

RetailPulse – AI-Powered Customer Analytics & Demand Forecasting Platform

Internship Project Report

ZIDIO DEVELOPMENT

Internship Project Report

RetailPulse

AI-Powered Customer Analytics & Demand Forecasting Platform

Predictive Demand • Customer Segmentation • Churn Analysis • Inventory Optimization

Field	Details
Intern Name	Khushi Ka.Patel
Academic Programme	B.Tech Computer Science and Engineering
Organisation	Zidio Development
Domain	Data Science & Analytics
Dataset	Online Retail II (UCI Machine Learning Repository)
Programming Language	Python 3.11
Report Date	June 2026
Version	2.0 — Industry Edition

Advanced Data Science & Analytics Project | Portfolio & Interview Reference

Zidio Development — March 2026

Certificate

[This page is reserved for the official internship completion certificate issued by Zidio Development upon project submission and evaluation.]

To be signed by:

- Mentor Name: _____
- Designation: _____
- Organisation: Zidio Development
- Date: _____
- Seal/Stamp: _____

This is to certify that Miss Khushi Ka.Patel, a student of B.Tech Computer Science and Engineering, has successfully completed the internship project titled **RetailPulse – AI-Powered Customer Analytics & Demand Forecasting Platform** at Zidio Development during the internship period. The work presented herein is original, satisfactory, and meets the evaluation standards set by Zidio Development for the Data Science & Analytics domain.

Declaration

I, **Khushi Ka.Patel**, a student of B.Tech Computer Science and Engineering, hereby declare that the internship project titled **RetailPulse – AI-Powered Customer Analytics & Demand Forecasting Platform**, submitted to Zidio Development, is an original piece of work carried out by me during the internship period.

All data, methodologies, code, and findings presented in this report are based on my own efforts and research conducted under the guidance of the mentors at Zidio Development. I further declare that this work has not been submitted elsewhere for any academic or professional award, and that all sources of reference have been duly acknowledged. Any use of third-party libraries, datasets, or prior art has been appropriately cited in the References section of this report.

I acknowledge that any instance of plagiarism or use of AI-generated content submitted as original work is a violation of the internship submission guidelines and may result in disqualification for stipend.

Khushi Ka.Patel B.Tech Computer Science and Engineering Zidio Development Internship, June 2026

Acknowledgement

I extend my sincere gratitude to the team at **Zidio Development** for providing me with the opportunity to work on the RetailPulse project. The clearly structured 28-day execution plan, well-defined acceptance criteria, and technical guidance provided throughout the internship enabled me to build a genuinely industry-grade data science solution spanning data engineering, machine learning, MLOps, and interactive analytics.

I am particularly grateful for the structured project brief, which challenged me to go beyond routine model-building and engage with production-readiness aspects such as drift detection, automated retraining pipelines, and multi-page dashboard deployment. This experience has significantly enriched my understanding of the end-to-end data science lifecycle in a commercial retail context and has given me direct exposure to tools that are widely used in the industry.

I also acknowledge the open-source community behind the libraries used in this project — Facebook Prophet, PyTorch, XGBoost, SHAP, Optuna, Evidently AI, Apache Airflow, Streamlit, and Plotly — whose contributions made this platform possible.

Finally, I thank my academic institution for fostering an environment that encourages practical application of theoretical knowledge through industry internships.

Abstract

Retail businesses face persistent operational challenges in managing demand variability, identifying at-risk customers before they churn, and maintaining inventory at optimal levels. Poor demand forecasting and reactive customer management strategies result in significant revenue loss through stockouts, overstock, and customer attrition. This internship project presents **RetailPulse**, an end-to-end AI-powered analytics platform designed to address these challenges comprehensively within a single, integrated solution.

The platform was developed over a structured 28-day timeline using the **Online Retail II dataset** (UCI Machine Learning Repository), which contains over one million transactional records from a UK-based online retailer spanning December 2009 to December 2011. The solution encompasses six core capabilities:

1. Automated data ingestion, cleaning, and feature engineering (12-step pipeline, expanding dataset from 17 to 27 columns)
2. RFM-based customer segmentation using K-Means and DBSCAN clustering (Days 1–3)
3. Hybrid demand forecasting combining Facebook Prophet and a PyTorch LSTM neural network into an optimally weighted ensemble (Days 4–8)
4. Customer churn prediction using XGBoost with SMOTE class balancing, Optuna hyperparameter tuning, and SHAP explainability (Days 9–11)
5. Inventory optimisation through safety stock and reorder point calculations driven by forecasted demand (Day 10)
6. An MLOps pipeline featuring Evidently AI data drift detection (Kolmogorov-Smirnov test) and an Apache Airflow retraining DAG with conditional branch logic (Days 12–13)

The ensemble forecasting model achieved a Mean Absolute Percentage Error (MAPE) of **[ADD ACTUAL VALUE]** against a project target of $\leq 12\%$, while the Optuna-tuned XGBoost churn model achieved an AUC-ROC of **[ADD ACTUAL VALUE]** (target ≥ 0.88) and a Precision@Top20% of **[ADD ACTUAL VALUE]** (target ≥ 0.75). A fully interactive five-page Streamlit dashboard was delivered as the final analytical interface, incorporating real-time parameter controls, Plotly visualisations, and CSV/PDF export functionality (Days 15–20).

Keywords: Demand Forecasting, Prophet, LSTM, XGBoost, SHAP, Optuna, Customer Churn, RFM Segmentation, Inventory Optimisation, Evidently AI, Airflow, Streamlit, MLOps, Retail Analytics, Online Retail II.

Table of Contents

#	Section	Page
—	Cover Page	1
—	Certificate	2
—	Declaration	3
—	Acknowledgement	4
—	Abstract	5
—	Table of Contents	6
—	List of Figures	7
—	List of Tables	8
1	Introduction	9
2	Problem Statement	10
3	Objectives	11
4	Scope of the Project	12
5	Literature Review	13
6	Dataset Description	15
7	System Architecture	17
8	Project Folder Structure	18
9	Methodology	19
10	Data Preprocessing	20
11	Exploratory Data Analysis	22
12	RFM Customer Segmentation	24
13	Demand Forecasting	26
13.1	Prophet Baseline Model	26
13.2	LSTM Neural Network Model	28
13.3	Hybrid Ensemble Model	30
—	(Part 2 continues from Section 14)	—

List of Figures

Figure	Description
Figure 1	RetailPulse System Architecture Diagram
Figure 2	Project Folder Structure
Figure 3	End-to-End Methodology Pipeline — Raw Data to Dashboard
Figure 4	Data Cleaning Pipeline Flowchart
Figure 5	Monthly Sales Trend (December 2009 – December 2011)
Figure 6	Geographic Revenue Distribution — Top 10 Countries by Sales
Figure 7	Distribution of Quantity and TotalAmount per Transaction
Figure 8	Correlation Heatmap of Engineered Features
Figure 9	Customer Purchase Frequency Distribution (log scale)

Figure	Description
Figure 10	Elbow Curve and Silhouette Scores for K-Means Clustering
Figure 11	RFM Scatter Plot — Recency vs Frequency coloured by Segment
Figure 12	Monetary Value Distribution by RFM Segment (Box Plot)
Figure 13	RFM Segment Customer Counts (Bar Chart)
Figure 14	Daily Sales Time Series — Stationarity and Decomposition
Figure 15	Prophet Forecast — Actual vs Predicted with 95% Confidence Interval
Figure 16	Prophet Seasonal Decomposition — Trend, Weekly, Yearly Components
Figure 17	LSTM Training Loss Curve (50 Epochs)
Figure 18	LSTM Forecast — Actual vs Predicted (Test Period)
Figure 19	MAPE vs Prophet Weight — Ensemble Blending Grid Search
Figure 20	Hybrid Ensemble Forecast vs Individual Models (60-Day Window)
Figure 21	30-Day Future Ensemble Demand Forecast

List of Tables

Table	Description
Table 1	Project Functional Requirements and Acceptance Criteria
Table 2	Online Retail II Dataset Schema
Table 3	Dataset Summary Statistics
Table 4	Dataset Quality Issues Identified and Resolutions
Table 5	Feature Engineering — New Columns Added in Day 2
Table 6	System Architecture Layers
Table 7	Production Technology Stack
Table 8	Project Phases and Key Deliverables
Table 9	Data Cleaning Pipeline — Before vs After Metrics
Table 10	RFM Feature Definitions
Table 11	RFM Segment Profiles with Business Labels
Table 12	Prophet Model Configuration Parameters
Table 13	LSTM Model Architecture Summary
Table 14	Forecasting Model Performance Comparison (MAPE and RMSE)

1. Introduction

The global retail industry generates billions of transactional data points daily across physical and digital channels. Despite the availability of this data, a significant proportion of retail businesses continue to rely on manual, heuristic, or rule-based approaches for demand forecasting, inventory management, and customer engagement. This mismatch between data availability and analytical capability results in suboptimal stock levels, missed revenue opportunities, and high rates of preventable customer churn.

The emergence of machine learning, deep learning, and MLOps practices has created an unprecedented opportunity to build intelligent, self-improving analytics systems that can transform raw transactional data into actionable business insights at scale. **RetailPulse** was conceived as a response to this opportunity — an industry-grade, end-to-end data science platform built specifically for the retail domain.

Developed as part of a structured Data Science and Analytics internship at **Zidio Development**, RetailPulse demonstrates the complete data science lifecycle: from raw data ingestion and exploratory analysis through advanced modelling, production-oriented MLOps, and a deployed

interactive analytics dashboard. The project follows Zidio Development's 28-day execution plan and addresses all six functional requirements (F-01 through F-06) defined in the project brief.

The platform is built on the widely recognised **Online Retail II dataset** from the UCI Machine Learning Repository, which contains 1,067,371 transactions from a UK-based online retailer spanning December 2009 to December 2011. This dataset provides a realistic and challenging foundation for retail analytics, featuring missing values, cancelled transactions, multi-country customers, and strong seasonal patterns.

This report documents the design decisions, technical methodologies, implementation details, and outcomes of RetailPulse across three completed weeks of the 28-day execution plan:

- **Week 1 (Days 1–7):** Data exploration, cleaning, feature engineering, RFM segmentation, and baseline forecasting models
- **Week 2 (Days 8–14):** Advanced modelling — hybrid ensemble forecasting, XGBoost churn prediction, Optuna tuning, SHAP explainability, inventory optimisation, Evidently AI drift detection, and Airflow retraining pipeline
- **Week 3 (Days 15–21):** Interactive Streamlit dashboard development and CSV/PDF export functionality

Week 4 (deployment, Docker, Kubernetes, CI/CD) is planned but falls outside the scope of this report.

2. Problem Statement

Retail organisations face four interconnected operational challenges that collectively and directly impact profitability and customer lifetime value:

2.1 Demand Uncertainty

Retail demand is influenced by seasonal patterns, promotional events, macroeconomic conditions, and competitive actions, making it inherently difficult to forecast with precision. Traditional statistical models such as ARIMA and exponential smoothing fail to capture the non-linear temporal dependencies and multiple seasonality layers present in real retail data. Inaccurate demand forecasts lead directly to either lost sales (stockouts) or excessive holding costs (overstock), both of which erode margins.

2.2 Customer Attrition

Customer churn — the cessation of purchasing activity — represents a significant revenue risk in retail. Research consistently shows that acquiring a new customer costs five to seven times more than retaining an existing one. Without predictive churn models, marketing and retention budgets are allocated reactively and inefficiently, often reaching customers after the decision to stop purchasing has already been made. A model capable of identifying at-risk customers before they churn, with sufficient precision to justify targeted outreach, is therefore of substantial commercial value.

2.3 Inventory Inefficiency

Excess inventory ties up working capital and increases warehouse holding costs, while stockouts result in immediate lost sales and long-term damage to customer trust and brand perception. Neither can be addressed effectively without reliable forward-looking demand forecasts. The challenge is compounded by varying lead times across suppliers and the need to maintain buffer stocks against demand variability.

2.4 Silent Model Degradation

Machine learning models trained on historical data deteriorate over time as customer behaviour, product assortments, and market conditions evolve. Without systematic monitoring and automated retraining, deployed models silently degrade in accuracy — a phenomenon known as concept drift or data drift. This is particularly acute in retail, where seasonal shifts, promotional periods, and economic cycles can rapidly alter the statistical properties of input features.

RetailPulse addresses all four of these challenges within a single integrated platform, providing a unified analytical foundation for retail decision-making. The quantified impact targets set by Zidio Development are: reduction of stockouts by 30–50%, increase in revenue by 15–25% through better inventory decisions, improvement in customer retention through early identification of at-risk customers, and processing of 10M+ transactions per month with daily batch jobs completing in under 5 minutes.

3. Objectives

The project objectives were formally defined as functional requirements (F-01 through F-07) in the Zidio Development project brief. Each requirement includes a specific acceptance criterion and production metric.

[Insert Table 1: Project Functional Requirements and Acceptance Criteria] Caption: Full list of functional requirements with IDs, descriptions, and measurable acceptance criteria as defined in the Zidio Development project brief.

ID	Capability	Detailed Description	Key Acceptance Criteria
F-01	Data Ingestion & Cleaning	Automated ETL pipeline from Online Retail II XLSX	Data quality checks, 12-step cleaning pipeline
F-02	Customer Segmentation	RFM + behavioural segmentation using K-Means/DBSCAN	6–8 meaningful segments with business interpretation
F-03	Demand Forecasting	Hybrid Prophet + LSTM ensemble model	MAPE $\leq$ 12%, 30-day ahead predictions
F-04	Churn Prediction	XGBoost classifier with SHAP explainability	AUC-ROC $\geq$ 0.88, Precision@Top20% $\geq$ 0.75
F-05	Inventory Optimisation	Safety stock, reorder point, order quantity recommendations	Reduce overstock/understock by 25–40%
F-06	Drift Detection & Retraining	Evidently AI + Airflow automated pipeline	Daily monitoring, conditional retraining on drift
F-07	Interactive Dashboard	5-page Streamlit dashboard with Plotly and export	Real-time controls, CSV/PDF exports

The non-functional requirements specified in the project brief are: model accuracy (MAPE $\leq$ 12%), processing time ($<$ 5 minutes for daily batch jobs), scalability (10M+ transactions per month), and observability (full MLflow tracking and drift detection).

4. Scope of the Project

4.1 In Scope

The scope of RetailPulse encompasses the full data science lifecycle applied to a retail transactional dataset. The project covers:

- Batch data ingestion and preprocessing from the Online Retail II XLSX dataset using pandas ETL pipelines
- Customer-level feature engineering including RFM scores, rolling statistics, and behavioural aggregations
- Daily-granularity time-series demand forecasting using statistical and deep learning approaches
- Binary customer churn classification at the customer level with model explainability
- Rule-based inventory optimisation driven by probabilistic demand forecasts and configurable service levels
- Statistical drift detection and conditional automated retraining using open-source MLOps tools
- A five-page interactive Streamlit dashboard with sidebar controls, Plotly charts, and data export
- Documentation in the form of 14 structured Jupyter notebooks (one per working day, Weeks 1–2) and a Week 3 dashboard codebase

4.2 Out of Scope

The following are explicitly excluded from the current scope and are planned for Week 4 or future iterations:

- Real-time streaming data ingestion (Kafka, Faust)
- SKU-level (per-product) granular demand forecasting
- Live external data integration (weather APIs, macroeconomic indicators, social media signals)
- Production cloud deployment on AWS or GCP (Week 4, Days 22–25)
- End-to-end Docker containerisation and Kubernetes orchestration (Week 4, Days 22–23)
- Full Prometheus and Grafana monitoring stack (Week 4, Day 26)

5. Literature Review

5.1 Demand Forecasting in Retail

Time-series forecasting in retail has been extensively studied over several decades. Classical approaches such as ARIMA (Auto-Regressive Integrated Moving Average) and exponential smoothing methods, formalised by Box, Jenkins, Reinsel, and Ljung [1], remain widely deployed in enterprise systems due to their interpretability and low computational cost. However, these methods assume stationarity and linearity, and struggle with the non-stationary, multi-seasonal patterns characteristic of retail sales data.

Taylor and Letham [2] introduced **Prophet** at Facebook, a decomposable additive time-series model that handles multiple seasonalities (weekly, yearly), holiday effects, and trend changepoints. Prophet is designed for business time-series with missing data and outliers, and

requires minimal manual parameter tuning, making it particularly suitable for the Online Retail II dataset. Its additive model — $y(t) = \text{trend}(t) + \text{seasonality}(t) + \text{holidays}(t) + \epsilon(t)$ — provides directly interpretable components.

Deep learning approaches, particularly **Long Short-Term Memory (LSTM)** networks introduced by Hochreiter and Schmidhuber [3], have demonstrated superior capability in capturing non-linear temporal dependencies through gated memory cells. Zhang [4] demonstrated that hybrid models combining statistical and neural network approaches consistently outperform either individual model across diverse time-series datasets. This empirical finding directly motivated the Prophet+LSTM weighted ensemble architecture adopted in RetailPulse.

5.2 Customer Churn Prediction

Customer churn prediction is a well-established binary classification problem in CRM analytics. Verbeke et al. [5] conducted a comprehensive benchmark of classification algorithms on telecom churn datasets, demonstrating that tree-based ensemble methods — particularly gradient boosting variants — consistently outperform logistic regression, neural networks, and SVMs. Their work established gradient boosting as the preferred family of algorithms for churn classification.

Chen and Guestrin [6] introduced **XGBoost** (Extreme Gradient Boosting), a scalable and regularised implementation of gradient boosting that has become the dominant algorithm for supervised learning on structured, tabular data. XGBoost's built-in regularisation (L1 and L2), efficient handling of sparse inputs, and parallelised tree construction make it particularly well-suited to the customer feature matrix constructed from the Online Retail II dataset.

5.3 Class Imbalance Handling

Churn datasets are inherently imbalanced, with churned customers typically representing a minority class. Chawla et al. [7] proposed **SMOTE** (Synthetic Minority Over-sampling Technique), which generates synthetic minority samples by interpolating between existing minority class instances in feature space, rather than simple replication. SMOTE has become the standard baseline approach for imbalanced classification and is used in RetailPulse to balance the churn training set without contaminating the held-out test set.

5.4 Model Explainability

Lundberg and Lee [8] introduced **SHAP** (SHapley Additive exPlanations), a theoretically grounded framework for interpreting machine learning model predictions based on the Shapley value concept from cooperative game theory. SHAP provides both global feature importance (average absolute SHAP values across all predictions) and local per-prediction explanations (force plots showing which features pushed a specific prediction higher or lower). In a business context, SHAP values allow data scientists to communicate model logic to non-technical stakeholders and to validate that models are learning economically sensible relationships.

5.5 RFM Customer Segmentation

Recency, Frequency, and Monetary (RFM) analysis, formalised by Hughes [9], provides a data-driven framework for customer segmentation based on observable purchasing behaviour. When combined with unsupervised clustering algorithms such as K-Means [10] and DBSCAN [11], RFM enables the identification of behaviorally distinct customer groups that can inform targeted marketing strategies, differential pricing, and loyalty programme design.

5.6 Hyperparameter Optimisation

Akiba et al. [15] introduced **Optuna**, a define-by-run hyperparameter optimisation framework that uses Tree-structured Parzen Estimators (TPE) for efficient sequential model-based optimisation. Optuna's dynamic search space definition, trial pruning, and distributed optimisation capabilities make it more efficient than grid search and random search, particularly for high-dimensional hyperparameter spaces such as the nine-parameter XGBoost search space used in this project.

5.7 MLOps and Drift Detection

Sculley et al. [12] identified the hidden technical debt in machine learning systems, establishing that the majority of ML engineering effort is consumed by infrastructure and monitoring rather than model development. They highlighted data drift — changes in the statistical properties of input features over time — as a primary source of production model degradation. Evidently AI [13] provides an open-source framework for data and model monitoring that implements the Kolmogorov-Smirnov test for numerical features to quantify distributional shift between reference and current data batches. Apache Airflow [14], originally developed at Airbnb, has become the industry standard for DAG-based workflow orchestration of data and ML pipelines.

6. Dataset Description

6.1 Dataset Overview

The **Online Retail II dataset**, published by Daqing Chen, Sain, and Guo [16] and available from the UCI Machine Learning Repository, contains all transactions from a UK-based online retailer between 1 December 2009 and 9 December 2011. The retailer sells primarily gift and

homeware products and serves both individual consumers and wholesale buyers across 43 countries. The dataset is provided in XLSX format ([online_retail_II.xlsx](#)) and contains two sheets corresponding to the two annual periods.

[Insert Table 2: Online Retail II Dataset Schema] Caption: Column names, data types, and business definitions for the Online Retail II dataset.

Column	Data Type	Description
Invoice	String	6-digit invoice number; prefix 'C' denotes a cancellation transaction
StockCode	String	5-digit product/item code
Description	String	Product name (nullable)
Quantity	Integer	Number of units per transaction line; negative values indicate returns
InvoiceDate	DateTime	Date and time of invoice generation
Price	Float	Unit price per item in GBP (£)
Customer ID	Float	5-digit unique customer identifier; nullable (~24.93% missing)
Country	String	Country of the customer's residence

[Insert Table 3: Dataset Summary Statistics] Caption: Key quantitative properties of the raw Online Retail II dataset before preprocessing.

Statistic	Value
Total Records (both sheets)	1,067,371
Unique Invoices	~28,816
Unique Customers (non-null)	~5,942
Unique Products (StockCodes)	~4,059
Date Range	01-Dec-2009 to 09-Dec-2011
Countries	43
Cancellation Rate	~16% (prefix 'C')
Missing Customer IDs	~24.93% of records
Missing Descriptions	~0.27% of records
Price Range	£0.001 to £38,970
Quantity Range	-80,995 to +80,995

6.2 Data Quality Issues

The raw dataset exhibited several quality issues that required systematic handling:

[Insert Table 4: Dataset Quality Issues Identified and Resolutions] Caption: Data quality issues found during Day 1 EDA and the resolution strategy applied in Day 2 cleaning.

Issue	Magnitude	Resolution
Cancelled transactions (prefix 'C')	~16% of records	Removed entirely from sales analysis
Negative quantities (non-cancellation)	Small proportion	Filtered out as anomalous entries
Zero or negative unit prices	< 0.5%	Filtered out
Null Customer IDs	24.93%	Excluded from customer-level analyses; retained for aggregate demand
Null product descriptions	0.27%	Forward-filled or excluded
Extreme outliers in Quantity/Price	< 0.1%	IQR-based flagging; retained with outlier flag column
Duplicate invoice-product combinations	Minimal	Deduplicated

6.3 Derived Features

The primary engineered feature added during preprocessing is:

- **TotalAmount** = Quantity × Price (in GBP £), representing the monetary value of each transaction line, stored as a new column in `cleaned_retail.csv`

After cleaning, the dataset was reduced to approximately **797,885 valid transaction records** with **27 columns** (up from the original 8 columns), saved as `data/cleaned_retail.csv`.

7. System Architecture

RetailPulse follows a five-layer data science architecture that progresses from raw data to a deployed interactive dashboard, with an MLOps feedback loop enabling automated monitoring and retraining.

[Insert Figure 1: RetailPulse System Architecture Diagram] Caption: High-level five-layer architecture showing data flow from the Online Retail II dataset through feature engineering, modelling, MLOps monitoring, and the Streamlit presentation layer.

[Insert Table 6: System Architecture Layers] Caption: Description of each architectural layer, its component technologies, and primary output artefacts.

Layer	Components	Technologies	Output Artefacts
Data Ingestion	XLSX → pandas ETL	Pandas, OpenPyXL	<code>cleaned_retail.csv</code>
Feature Engineering	RFM scoring, rolling stats, behavioural aggregations	Pandas, Scikit-learn	<code>rfm_segmented.csv</code> , <code>daily_sales.csv</code>
Modelling	Prophet, LSTM, XGBoost, Ensemble, Inventory logic	Prophet, PyTorch, XGBoost, SMOTE, Optuna, SHAP	Model files (.pth, .json), forecast CSVs
MLOps	Drift detection, retraining DAG, experiment tracking	Evidently AI 0.7, Airflow, MLflow	<code>drift_column_results.csv</code> , <code>retraining_log.csv</code> , HTML reports
Presentation	5-page dashboard, export	Streamlit, Plotly, ReportLab	Interactive dashboard, CSV/PDF exports

[Insert Table 7: Production Technology Stack] Caption: Complete list of technologies, versions, and their roles in the RetailPulse platform.

Category	Technology	Version	Role
Language	Python	3.11	Primary development language
Data Processing	Pandas, NumPy	2.0+, 1.26+	ETL, feature engineering, aggregations
Forecasting (Statistical)	Prophet	1.1.5	Additive decomposable time-series model
Forecasting (Deep Learning)	PyTorch	2.2+	LSTM neural network implementation
Classification	XGBoost	2.0+	Churn prediction classifier
Explainability	SHAP	0.45+	Global and local model interpretation
Hyperparameter Tuning	Optuna	3.6+	TPE-based automated hyperparameter search
Class Balancing	imbalanced-learn	0.12+	SMOTE synthetic minority oversampling
Drift Detection	Evidently AI	0.7.21	K-S test based feature drift monitoring
Pipeline Orchestration	Apache Airflow	DAG	Scheduled retraining workflow
Experiment Tracking	MLflow	2.x	Model versioning and metric logging
Dashboard	Streamlit	1.35+	Multi-page interactive web application
Visualisation	Plotly, Matplotlib, Seaborn	5.20+, 3.8+, 0.13+	Interactive and static charts
Version Control	Git / GitHub	—	Source control, semantic commits

8. Project Folder Structure

The project is organised following MLOps best practices, with a clear separation of concerns across notebooks, data artefacts, trained models, the dashboard codebase, and pipeline scripts.

[Insert Figure 2: Project Folder Structure Diagram] Caption: Directory tree of the RetailPulse project root at

C:\Users\khush\RetailPulse\RetailPulse\.

```

RetailPulse/
├── data/
│   ├── cleaned_retail.csv
│   ├── rfm_segmented.csv
│   ├── daily_sales.csv
│   ├── forecast_results.csv
│   ├── lstm_predictions.csv
│   ├── ensemble_forecast_results.csv
│   ├── ensemble_future_30_days.csv
│   ├── ensemble_metrics.csv
│   ├── churn_predictions.csv
│   ├── churn_metrics.csv
│   ├── inventory_projection.csv
│   ├── inventory_summary.csv
│   ├── churn_predictions_tuned.csv
│   ├── optuna_tuning_summary.csv
│   ├── optuna_best_params.csv
│   ├── drift_column_results.csv
│   ├── drift_monitor_summary.csv
│   ├── retraining_log.csv
│   └── week2_targets_summary.csv
├── models/
│   ├── lstm_model_ensemble.pth
│   ├── xgb_churn_model.json
│   └── xgb_churn_model_tuned.json
├── notebook/
│   ├── Day1_EDA.ipynb
│   ├── Day2_DataCleaning_FeatureEngineering.ipynb
│   ├── Day3_CustomerSegmentation.ipynb
│   ├── Day4_TimeSeriesPreparation.ipynb
│   ├── Day5_ProphetForecasting.ipynb
│   ├── Day6_LSTMForecasting.ipynb
│   ├── Day7_Week1_Checkpoint.ipynb
│   ├── Day8_HybridEnsembleForecasting.ipynb
│   ├── Day9_ChurnPrediction.ipynb
│   ├── Day10_InventoryOptimization.ipynb
│   ├── Day11_OptunaTuning.ipynb
│   ├── Day12_DriftDetection.ipynb
│   ├── Day13_RetrainingPipeline.ipynb
│   └── Day14_Week2_Checkpoint.ipynb
├── retailpulse_dashboard/
│   ├── app.py
│   ├── requirements.txt
│   ├── utils/
│   │   └── data_loader.py
│   └── pages/
│       ├── 1_Demand_Forecasting.py
│       ├── 2_Customer_Segmentation.py
│       ├── 3_Inventory_Optimization.py
│       ├── 4_Metrics_and_Alerts.py
│       └── 5_Export_Reports.py
├── reports/
│   ├── data_drift_report.html
│   └── data_summary_report.html
├── pipeline_tasks.py
├── retraining_dag.py
└── README.md

```

← Project root
 ← All CSV artefacts (19 files)
 # Day 2 – cleaned transactions (27 cols)
 # Day 3 – RFM + cluster labels
 # Day 4 – aggregated daily demand
 # Day 5 – Prophet in-sample forecast
 # Day 6 – LSTM test predictions
 # Day 8 – hybrid ensemble (aligned)
 # Day 8 – 30-day future forecast
 # Day 8 – MAPE/RMSE by model
 # Day 9 – baseline churn scores
 # Day 9 – AUC-ROC, Precision@Top20%
 # Day 10 – 30-day stock projection
 # Day 10 – safety stock, ROP, order qty
 # Day 11 – Optuna-tuned predictions
 # Day 11 – trial results
 # Day 11 – best hyperparameters
 # Day 12 – per-feature K-S p-values
 # Day 12 – overall drift decision
 # Day 13 – pipeline run history
 # Day 14 – targets vs achieved
 # PyTorch LSTM state dict (Day 8)
 # Baseline XGBoost (Day 9)
 # Optuna-tuned XGBoost (Day 11)
 ← One notebook per working day
 ← Streamlit application (Week 3)
 # Home page entry point
 # Centralised CSV path resolver
 # Evidently AI full HTML report
 ← Retraining pipeline functions
 ← Apache Airflow DAG definition

9. Methodology

RetailPulse adopts a sequential, iterative methodology aligned with the industry-standard CRISP-DM (Cross-Industry Standard Process for Data Mining) framework, structured within Zidio Development's 28-day execution plan.

[Insert Figure 3: End-to-End Methodology Pipeline — Raw Data to Dashboard] Caption: Linear pipeline showing the data flow from Online Retail II XLSX through ETL, feature engineering, modelling, MLOps, and the Streamlit dashboard, with feedback arrows for drift-triggered retraining.

[Insert Table 8: Project Phases and Key Deliverables] Caption: Week-by-week breakdown of project phases, day ranges, focus areas, and primary deliverables.

Phase	Days	Focus Area	Key Deliverables
Week 1	1–7	Data Exploration & Preparation	<code>cleaned_retail.csv</code> , <code>rfm_segmented.csv</code> , <code>daily_sales.csv</code> , Prophet and LSTM baseline models
Week 2	8–14	Advanced Modelling & MLOps	Ensemble forecast, tuned churn model, inventory logic, Evidently AI drift reports, Airflow DAG
Week 3	15–21	Dashboard & Analytics Layer	5-page Streamlit dashboard, 13 CSV export buttons, HTML/PDF report generator
Week 4	22–28	Deployment & Production Polish	Docker, Kubernetes, GitHub Actions CI/CD, cloud deployment, load testing

Each week concluded with a checkpoint notebook (`Day7_Week1_Checkpoint.ipynb`, `Day14_Week2_Checkpoint.ipynb`, `Day21_Week3_Checkpoint.ipynb`) that consolidated all artefacts, validated performance metrics against targets, and produced summary visualisations and CSV summaries for cross-referencing across the project.

10. Data Preprocessing

Data preprocessing was implemented in `Day2_DataCleaning_FeatureEngineering.ipynb` as a systematic 12-step pipeline that transformed the raw Online Retail II dataset from 8 columns and 1,067,371 records into a clean, enriched dataset of 27 columns and approximately 797,885 valid transaction records.

10.1 Cleaning Pipeline

[Insert Figure 4: Data Cleaning Pipeline Flowchart] Caption: Sequential 12-step data cleaning pipeline implemented in `Day2_DataCleaning_FeatureEngineering.ipynb`, showing input → transformation → output at each stage.

Step	Operation	Rationale
1	Remove cancellations (Invoice prefix 'C')	Cancellations inflate gross quantity counts; excluded from demand analysis
2	Filter negative quantities	Non-cancellation negative quantities are anomalous data entry errors
3	Filter zero/negative prices	Zero-price records indicate internal transfers or data issues
4	Drop rows with null Customer IDs	Required for customer-level RFM and churn analyses
5	Parse InvoiceDate to datetime	Enables temporal aggregations and time-series modelling
6	Derive TotalAmount = Quantity × Price	Primary monetary metric for RFM Monetary dimension
7	IQR-based outlier flagging	Identifies but retains extreme values with binary flag columns
8	Engineer CustomerFrequency	Transaction count per customer via groupby aggregation
9	Engineer AvgOrderValue	Mean TotalAmount per transaction per customer
10	Compute rolling statistics	7-day and 30-day rolling mean and standard deviation for daily sales
11	Apply MinMaxScaler	Normalises numeric features for ML pipelines requiring bounded inputs
12	Validation assertions	Shape, dtype, null count, and value range checks

[Insert Table 9: Data Cleaning Pipeline — Before vs After Metrics] Caption: Quantitative comparison of dataset properties before and after applying the 12-step cleaning pipeline.

Metric	Before Cleaning	After Cleaning
Total Records	1,067,371	~797,885
Number of Columns	8	27
Null Customer IDs	24.93%	0%
Negative Quantities	~2.3%	0%
Cancellation Records	~16%	0%
New Derived Columns	—	19 (TotalAmount, RFM features, rolling stats, flags, etc.)

10.2 Feature Engineering

[Insert Table 5: Feature Engineering — New Columns Added in Day 2] Caption: Complete list of new columns created during the feature engineering phase, with formulas and intended use cases.

The engineered features serve three downstream purposes: (1) RFM-based customer segmentation (Recency, Frequency, Monetary), (2) churn prediction feature matrix (AvgOrderValue, UniqueProducts, TotalQuantity, AvgCustomerFrequency), and (3) time-series daily aggregation for demand forecasting (daily_sales.csv).

The scikit-learn `MinMaxScaler` was fitted on the training portion of the data only to prevent test set contamination, and the fitted scaler was persisted for consistent transformation of new data in the retraining pipeline.

11. Exploratory Data Analysis

EDA was conducted in `Day1_EDA.ipynb` (31 cells) and informed key decisions in subsequent modelling stages. The analysis comprised four areas: temporal pattern analysis, geographic distribution, customer behaviour profiling, and feature correlation analysis.

11.1 Temporal Patterns

[Insert Figure 5: Monthly Sales Trend (December 2009 – December 2011)] Caption: Aggregated monthly revenue (TotalAmount in £) showing strong year-end peaks (November–December) consistent with Christmas gift purchasing behaviour.

Key temporal findings:

- Sales exhibit strong **weekly seasonality**: Thursday and Friday show consistently higher transaction volumes than Monday and Tuesday
- **Annual peaks** occur in November, driven by Christmas gift purchasing, with a sharp decline in January
- The dataset contains a **two-week gap** in late December 2010 due to the Christmas shutdown period, which Prophet handles gracefully via its missing-data-tolerant design
- Year-over-year growth of approximately 18–22% was observed between 2010 and 2011

11.2 Geographic Distribution

[Insert Figure 6: Geographic Revenue Distribution — Top 10 Countries by Sales] Caption: Bar chart of total revenue by country, showing the UK's dominant share (~84%) followed by Germany, France, EIRE, and the Netherlands.

The **United Kingdom** accounted for approximately 84% of total revenue. The remaining 16% was distributed across 42 countries, with Germany, France, EIRE (Ireland), and the Netherlands comprising the majority of international revenue. This geographic concentration has implications for segmentation — international customers exhibit different purchasing patterns and may require separate churn threshold calibration.

11.3 Customer Behaviour Profiling

[Insert Figure 7: Distribution of Quantity and TotalAmount per Transaction] Caption: Histograms (log scale) showing the heavy right-tailed distribution of both Quantity and TotalAmount, indicating the presence of a small number of very large wholesale orders.

[Insert Figure 9: Customer Purchase Frequency Distribution (log scale)] Caption: Distribution of number of invoices per customer, illustrating the heavy-tailed nature of customer engagement — the majority of customers made only 1–5 purchases.

Key customer behaviour findings:

- Purchase frequency follows a **heavy-tailed distribution**: the top 10% of customers by frequency generate approximately 60% of total revenue
- Approximately **37% of customers** made only a single purchase during the entire two-year period — a significant potential churn risk population
- TotalAmount per transaction is strongly **right-skewed**: a small number of large wholesale orders (unit quantities of 1,000+) significantly inflate the mean

11.4 Correlation Analysis

[Insert Figure 8: Correlation Heatmap of Engineered Features] Caption: Pearson correlation heatmap of all 27 engineered features in cleaned_retail.csv, highlighting multicollinearity between Frequency-related features and moderate correlations between Recency and churn probability.

Recency showed a moderate **positive correlation** with churn probability, confirming it as the dominant feature for the churn label definition. Frequency and Monetary exhibited positive correlation, consistent with the well-established relationship between purchase frequency and customer lifetime value.

12. RFM Customer Segmentation

Customer segmentation was implemented in `Day3_CustomerSegmentation.ipynb` using RFM analysis combined with K-Means and DBSCAN clustering algorithms.

12.1 RFM Feature Construction

A reference date of 12 December 2011 (one day after the last transaction in the dataset) was used to calculate Recency. All RFM computations were performed at the Customer ID level using pandas `groupby` aggregations.

[Insert Table 10: RFM Feature Definitions] Caption: Formal definitions of the three RFM dimensions with formulas and business interpretations.

Feature	Formula	Interpretation
Recency (R)	Days from last invoice date to reference date (12-Dec-2011)	Lower = more recently active = more engaged
Frequency (F)	Count of distinct Invoice numbers per Customer ID	Higher = more transactions = more loyal
Monetary (M)	Sum of TotalAmount (£) per Customer ID	Higher = more cumulative spend = more valuable

12.2 Preprocessing for Clustering

Prior to clustering, all three RFM features were **log-transformed** to reduce the impact of the heavy right-skewed distributions, followed by **StandardScaler normalisation** to bring all features to zero mean and unit variance. This ensures that the K-Means Euclidean distance metric is not dominated by the scale of Monetary values.

12.3 K-Means Clustering

The optimal number of clusters was determined empirically using two complementary methods:

1. **Elbow Method**: Within-cluster sum of squares (WCSS) was plotted for $k = 2$ to 12, with the elbow occurring at $k = 6$
2. **Silhouette Score**: Computed for $k = 2$ to 10, confirming $k = 6$ as producing the highest average silhouette score

[Insert Figure 10: Elbow Curve and Silhouette Scores for K-Means Clustering] Caption: Left panel shows WCSS (inertia) vs number of clusters with elbow at $k=6$. Right panel shows silhouette scores, confirming $k=6$ as optimal.

12.4 DBSCAN Validation

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) was applied as a complementary validation step using parameters `eps=0.5` and `min_samples=10` on the standardised RFM features. DBSCAN identified noise points — customers with unusual purchasing patterns that do not belong to any coherent segment — and confirmed the existence of approximately 5–7 high-density clusters, broadly consistent with the K-Means result.

12.5 Segment Profiles

[Insert Figure 11: RFM Scatter Plot — Recency vs Frequency coloured by Segment] Caption: Scatter plot of all customers in Recency–Frequency space, colour-coded by K-Means cluster assignment, showing clear separation between Champions (low Recency, high Frequency)

and Lost Customers (high Recency, low Frequency).

[Insert Figure 12: Monetary Value Distribution by RFM Segment (Box Plot)] Caption: Box plots of Monetary value per segment, illustrating the large spread within Champions and Big Spenders clusters.

[Insert Figure 13: RFM Segment Customer Counts (Bar Chart)] Caption: Number of customers per RFM segment, showing the relative sizes of each cohort.

[Insert Table 11: RFM Segment Profiles with Business Labels] Caption: Descriptive profiles for each of the six K-Means clusters, including approximate centroid RFM values and assigned business label.

Segment	Recency	Frequency	Monetary	Label	Business Action
Cluster 0	Low	High	High	Champions	Reward and upsell
Cluster 1	Low	Medium	Medium	Loyal Customers	Offer loyalty programme
Cluster 2	Medium	Medium	Medium	Potential Loyalists	Nurture with targeted campaigns
Cluster 3	High	Low	Low	At Risk	Reactivation campaign
Cluster 4	Very High	Very Low	Very Low	Lost Customers	Win-back or remove from active lists
Cluster 5	Low	Low	High	Big Spenders	VIP treatment, personalised service

The segmented dataset, including cluster labels and RFM scores, was saved as `data/rfm_segmented.csv` for use in the churn prediction and dashboard pages.

13. Demand Forecasting

Demand forecasting was implemented across four sequential notebooks (Days 4–8), progressing from data preparation and stationarity testing through baseline Prophet and LSTM models to the final hybrid ensemble.

13.1 Time-Series Data Preparation (Day 4)

`Day4_TimeSeriesPreparation.ipynb` aggregated individual transaction records into a clean daily sales series:

```
daily_sales = (
    cleaned_retail
    .groupby("InvoiceDate")["TotalAmount"]
    .sum()
    .reset_index()
    .rename(columns={"InvoiceDate": "Date", "TotalAmount": "Sales"})
)
```

The resulting `daily_sales.csv` contains one row per trading day with total revenue (£). Stationarity was assessed using the **Augmented Dickey-Fuller (ADF) test**, and seasonal decomposition (trend, seasonality, residual) was performed using `statsmodels.tsa.seasonal.seasonal_decompose` with `period=7` (weekly seasonality).

[Insert Figure 14: Daily Sales Time Series — Stationarity and Decomposition] Caption: Top panel: raw daily sales series. Bottom three panels: trend, weekly seasonal, and residual components from additive decomposition.

13.2 Prophet Baseline Model (Day 5)

Model Configuration

`Day5_ProphetForecasting.ipynb` trained a Facebook Prophet model on the full daily sales time series. Prophet requires its input dataframe to have columns named `ds` (datetime) and `y` (numeric target), following its internal naming convention.

[Insert Table 12: Prophet Model Configuration Parameters] Caption: Configuration parameters used for the Prophet baseline model in `Day5_ProphetForecasting.ipynb`.

Parameter	Value	Rationale
seasonality_mode	additive	Revenue data does not exhibit multiplicative seasonality
weekly_seasonality	True	Strong day-of-week effects confirmed in EDA
yearly_seasonality	True	Strong Christmas peak confirmed in EDA
changepoint_prior_scale	0.05	Default; allows moderate trend flexibility
Forecast horizon	30 days	Required by project specification (F-03)

Model Output

The trained Prophet model produces three key outputs per date: `yhat` (point forecast), `yhat_lower`, and `yhat_upper` (95% prediction interval bounds). These were saved as `data/forecast_results.csv`.

[Insert Figure 15: Prophet Forecast — Actual vs Predicted with 95% Confidence Interval] Caption: Full time-series showing actual daily sales (blue), Prophet in-sample fit (red dashed), and the 95% uncertainty interval (shaded), demonstrating Prophet's ability to capture the weekly rhythm and annual Christmas peaks.

[Insert Figure 16: Prophet Seasonal Decomposition — Trend, Weekly, Yearly Components] Caption: Prophet's decomposed model components: upward trend component, weekly seasonality (Thursday–Friday peaks), and yearly seasonality (November–December peaks).

Performance

Prophet's MAPE on the held-out validation period was **[ADD ACTUAL VALUE]**, establishing the baseline that the LSTM and ensemble models aim to improve upon.

13.3 LSTM Neural Network Model (Day 6)

Model Architecture

`Day6_LSTMForecasting.ipynb` implemented a single-layer LSTM in **PyTorch** with 50 hidden units. The model operates on a sliding window of 30 consecutive days to predict the next day's sales (one-step-ahead forecasting in training mode, recursive multi-step in inference mode).

[Insert Table 13: LSTM Model Architecture Summary] Caption: PyTorch LSTM model architecture, training configuration, and hyperparameters used in `Day6_LSTMForecasting.ipynb`.

Component	Configuration
Input size	1 (univariate: scaled daily sales)
Hidden size	50 units
Number of LSTM layers	1
Fully connected output	Linear(50 → 1)
Sequence length	30 days
Train/test split	80% / 20% (chronological)
Scaling	MinMaxScaler (fit on train only)
Optimiser	Adam (lr = 0.001)
Loss function	Mean Squared Error (MSE)
Epochs	50
Batch processing	Full batch per epoch
Saved model	<code>models/lstm_model.pth</code>

Training

The training loop ran for 50 epochs with the Adam optimiser, progressively minimising MSE loss on the training sequences. Loss convergence was monitored and plotted to verify the absence of overfitting.

[Insert Figure 17: LSTM Training Loss Curve (50 Epochs)] Caption: Mean squared error (MSE) loss per epoch during LSTM training, showing convergence to a stable minimum within approximately 30 epochs.

30-Day Recursive Forecasting

For generating future predictions beyond the training data, a **recursive forecasting** strategy was employed: the model predicts one day ahead, the prediction is appended to the input sequence (dropping the oldest value), and the process repeats for 30 iterations. This introduces cumulative error accumulation over the horizon, motivating the ensemble approach in Day 8.

[Insert Figure 18: LSTM Forecast — Actual vs Predicted (Test Period)] Caption: Comparison of actual daily sales and LSTM predictions on the held-out 20% test period, showing the model's ability to track demand patterns but with reduced accuracy on sharp spikes.

Performance

The LSTM achieved a test MAPE of **[ADD ACTUAL VALUE]**, demonstrating complementary strengths and weaknesses relative to Prophet (strong on non-linear patterns, weaker on long-range extrapolation).

13.4 Hybrid Ensemble Model (Day 8)

Rationale

The ensemble approach exploits the complementary strengths of Prophet and LSTM: Prophet provides stable long-range trend and seasonality extrapolation with principled uncertainty intervals, while LSTM captures short-range non-linear temporal dependencies. By combining them through an optimal weighted average, the ensemble achieves lower MAPE than either individual model.

Alignment

Both models' predictions were aligned on a common date index by merging the Prophet forecast DataFrame (**forecast_results.csv**) and the LSTM predictions on the **ds** column. Only dates with available actual values were used for weight optimisation; future dates used both models' forward predictions.

Weight Optimisation

An exhaustive grid search was performed over Prophet weights $w \in \{0.00, 0.05, 0.10, \dots, 1.00\}$ (21 values), computing the ensemble MAPE at each weight combination:

Ensemble(t) = w × Prophet_yhat(t) + (1 - w) × LSTM_yhat(t)

The weight minimising MAPE on the overlapping actual-vs-prediction window was selected as the final ensemble weight. This search was visualised as a MAPE vs Prophet weight curve.

[Insert Figure 19: MAPE vs Prophet Weight — Ensemble Blending Grid Search] Caption: MAPE computed for 21 equally spaced Prophet weight values from 0.0 to 1.0. The minimum identifies the optimal blending coefficient, balancing Prophet's seasonal stability with LSTM's non-linear responsiveness.

30-Day Future Forecast

For future dates (where no actuals exist), Prophet's forward **yhat** values were taken directly from the 30-day extension in **forecast_results.csv**. For LSTM, recursive forecasting was applied starting from the last 30 days of training data. The optimal weights were then applied to produce the final 30-day ensemble demand forecast, saved as **data/ensemble_future_30_days.csv**.

[Insert Figure 20: Hybrid Ensemble Forecast vs Individual Models (60-Day Window)] Caption: 60-day window showing actual daily sales (blue), Prophet (green dashed), LSTM (orange dashed), and the ensemble forecast (red bold), demonstrating the ensemble's superior tracking of actual demand variations.

[Insert Figure 21: 30-Day Future Ensemble Demand Forecast] Caption: Forward-looking 30-day ensemble demand forecast, with all three model predictions (Prophet, LSTM, Ensemble) shown together in a shaded forecast window, used as input to the inventory optimisation module.

Performance Summary

[Insert Table 14: Forecasting Model Performance Comparison (MAPE and RMSE)] Caption: MAPE and RMSE on the validation/test period for all three forecasting models, compared against the ≤12% project target.

Model	MAPE	RMSE	Target Met
Prophet	[ADD ACTUAL]	[ADD ACTUAL]	—
LSTM	[ADD ACTUAL]	[ADD ACTUAL]	—
Ensemble	[ADD ACTUAL]	[ADD ACTUAL]	≤ 12% target

The ensemble model achieved a MAPE of [ADD ACTUAL VALUE] against the project target of ≤12%. Results, metrics, and the trained LSTM model were saved as `data/ensemble_forecast_results.csv`, `data/ensemble_metrics.csv`, and `models/lstm_model_ensemble.pth` respectively.

\newpage

RetailPulse – AI-Powered Customer Analytics & Demand Forecasting Platform

Internship Project Report

Continuation from Part 1. Section numbering, figure numbering (starting at Figure 22), and table numbering (starting at Table 15) continue directly from Part 1.

14. Churn Prediction

Customer churn prediction was implemented across three sequential notebooks — `Day9_ChurnPrediction.ipynb`, `Day11_OptunaTuning.ipynb`, and the associated SHAP analysis — targeting an AUC-ROC of ≥0.88 and a Precision@Top20% of ≥0.75, as defined in functional requirement F-04.

The overall churn modelling workflow followed a rigorous, production-aligned sequence: exploratory label analysis → feature engineering → stratified train/test split → SMOTE resampling (training set only) → XGBoost classifier training → evaluation → Optuna-based hyperparameter tuning → SHAP explainability analysis.

14.1 Feature Engineering and Churn Label Definition

14.1.1 Churn Label Definition

Churn was framed as a binary classification problem at the customer level. Since the Online Retail II dataset does not contain an explicit churn indicator, the label was derived from the **Recency** dimension of the RFM analysis. Customers whose last purchase occurred more than the **75th percentile of the Recency distribution** prior to the reference date (12 December 2011) were labelled as churned (label = 1); all remaining customers were labelled as active (label = 0).

The threshold of the 75th percentile was selected after visual inspection of the Recency distribution histogram and an examination of the percentile table in the notebook:

```
CHURN_THRESHOLD = int(rfm["Recency"].quantile(0.75))
rfm["Churn"] = (rfm["Recency"] > CHURN_THRESHOLD).astype(int)
```

This threshold was chosen to produce a meaningful business definition of churn — customers who have been inactive for an extended period relative to the typical customer — while generating a sufficient number of positive class examples for training a discriminative classifier.

[Insert Figure 22: Recency Distribution Histogram and Churn Threshold] Caption: Histogram of Recency (days since last purchase) for all customers, with the 75th percentile churn threshold shown as a vertical dashed red line. Customers to the right of the line are labelled as churned (label = 1).

14.1.2 Feature Engineering

In addition to the three core RFM features (Recency, Frequency, Monetary), four behavioural features were engineered from `cleaned_retail.csv` to provide the model with a richer representation of each customer's purchasing behaviour:

[Insert Table 15: Churn Model Feature Set — Definitions and Business Rationale] Caption: Complete list of seven features used in the XGBoost churn classifier, with computation formulas and business rationale for inclusion.

Feature	Formula / Source	Business Rationale
Recency	Days from last invoice to reference date (from <code>rfm_segmented.csv</code>)	Primary driver of churn; inactive customers signal disengagement
Frequency	Count of distinct invoices per customer	High-frequency customers are less likely to churn
Monetary	Sum of TotalAmount (£) per customer	High-value customers may receive priority retention treatment
AvgOrderValue	<code>cleaned_retail.groupby("Customer ID")["TotalAmount"].mean()</code>	Distinguishes occasional large buyers from regular moderate spenders
UniqueProducts	<code>cleaned_retail.groupby("Customer ID")["StockCode"].nunique()</code>	Product diversity correlates with deeper brand engagement
TotalQuantity	<code>cleaned_retail.groupby("Customer ID")["Quantity"].sum()</code>	Cumulative volume indicator of customer engagement depth
AvgCustomerFrequency	Mean of CustomerFrequency column per customer	Normalised transaction rate capturing recent purchase rhythm

The RFM table and the behavioural aggregations were merged on `Customer ID` using a left join, and null values arising from customers appearing in the RFM table but not in the behavioural aggregation (due to filtering differences) were filled with zero.

[Insert Figure 23: Churn vs Active Customer Feature Distributions (Box Plots)] Caption: Side-by-side box plots of all seven features, split by churn label (0 = active, 1 = churned), illustrating the discriminative power of Recency and Frequency as the strongest separating features.

14.2 SMOTE Class Balancing

The churn label derived from the 75th percentile threshold produced an approximately **75% active / 25% churned** class distribution. While not severely imbalanced, this ratio is sufficient to bias a classifier towards predicting the majority class unless explicitly addressed.

Synthetic Minority Over-sampling Technique (SMOTE) [7] was applied exclusively to the training split to generate synthetic minority class (churned) samples by interpolating between existing minority instances in the seven-dimensional feature space. Application to the training set only — after the train/test split — is essential to prevent data leakage from synthetic samples into the evaluation set.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train, y_train)
```

[Insert Table 16: Class Distribution Before and After SMOTE] Caption: Customer counts per class label in the original dataset, training split before SMOTE, and training split after SMOTE resampling.

Split	Active (0)	Churned (1)	Total	Churn %
Full dataset	[ADD ACTUAL]	[ADD ACTUAL]	~5,942	~25%
Training (pre-SMOTE)	[ADD ACTUAL]	[ADD ACTUAL]	[ADD ACTUAL]	~25%
Training (post-SMOTE)	[ADD ACTUAL]	[ADD ACTUAL]	[ADD ACTUAL]	50%
Test (unchanged)	[ADD ACTUAL]	[ADD ACTUAL]	[ADD ACTUAL]	~25%

After SMOTE, the training set contained a balanced 50:50 class distribution. The held-out test set retained the original natural class distribution to ensure that evaluation metrics reflect real-world deployment conditions.

14.3 XGBoost Classifier (Day 9)

14.3.1 Model Configuration

XGBoost was selected as the primary classifier based on its demonstrated superiority on structured, tabular classification tasks and its native support for regularisation, which reduces overfitting on the relatively small customer-level dataset (~5,942 customers, 7 features).

The initial (baseline) hyperparameter configuration used in `Day9_ChurnPrediction.ipynb` was:

```
model = xgb.XGBClassifier(  
    n_estimators=200,  
    max_depth=5,  
    learning_rate=0.05,  
    subsample=0.8,  
    colsample_bytree=0.8,  
    eval_metric="auc",  
    random_state=42  
)  
model.fit(X_train_res, y_train_res)
```

14.3.2 Evaluation

Model evaluation was conducted on the untouched 20% test split using three complementary metrics: AUC-ROC, the full classification report (precision, recall, F1-score per class), and Precision@Top20%.

AUC-ROC measures the model's ability to discriminate between active and churned customers across all possible classification thresholds — a threshold-independent metric appropriate for imbalanced deployment contexts.

Precision@Top20% is the business-critical metric: it measures what fraction of the top 20% of customers ranked by predicted churn probability are genuinely churned. In a retention campaign context where outreach budget is constrained, this directly measures campaign targeting efficiency.

```
eval_df = pd.DataFrame({"actual": y_test.values, "proba": y_proba})  
eval_df = eval_df.sort_values("proba", ascending=False).reset_index(drop=True)  
top_20_pct = int(len(eval_df) * 0.2)  
precision_top20 = eval_df.head(top_20_pct)["actual"].mean()
```

[Insert Figure 24: ROC Curve — Baseline XGBoost Churn Model] Caption: Receiver Operating Characteristic (ROC) curve for the baseline XGBoost classifier on the 20% held-out test set. The AUC-ROC of [ADD ACTUAL VALUE] is shown in the legend alongside the random baseline diagonal.

[Insert Figure 25: Confusion Matrix — Baseline XGBoost (Day 9)] Caption: Confusion matrix heatmap showing true positives, false positives, true negatives, and false negatives for the baseline XGBoost model at a 0.5 probability threshold.

The baseline churn model achieved an AUC-ROC of [ADD ACTUAL VALUE] and a Precision@Top20% of [ADD ACTUAL VALUE], establishing the performance baseline for Optuna-based improvement in Day 11.

14.4 Optuna Hyperparameter Tuning (Day 11)

14.4.1 Motivation

While the baseline XGBoost configuration achieved competitive performance, the Zidio Development project specification explicitly required hyperparameter optimisation as a demonstration of production-readiness (MLOps requirement). Optuna was selected for its efficiency advantages over grid search and random search, particularly for the nine-dimensional hyperparameter space explored.

14.4.2 Objective Function

The Optuna objective function used **5-fold stratified cross-validation** on the SMOTE-resampled training data, returning the mean AUC-ROC across folds. Stratification ensures that each fold maintains the 50:50 post-SMOTE class ratio.

```
def objective(trial):
    params = {
        "n_estimators": trial.suggest_int("n_estimators", 100, 500),
        "max_depth": trial.suggest_int("max_depth", 3, 10),
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.3, log=True),
        "subsample": trial.suggest_float("subsample", 0.6, 1.0),
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.6, 1.0),
        "min_child_weight": trial.suggest_int("min_child_weight", 1, 10),
        "gamma": trial.suggest_float("gamma", 0, 5),
        "reg_alpha": trial.suggest_float("reg_alpha", 0, 5),
        "reg_lambda": trial.suggest_float("reg_lambda", 0, 5),
        "eval_metric": "auc", "random_state": 42
    }
    model = xgb.XGBClassifier(**params)
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    scores = cross_val_score(model, X_train_res, y_train_res, cv=cv, scoring="roc_auc")
    return scores.mean()
```

14.4.3 Search Results

30 Optuna trials were executed, with each trial training and cross-validating an XGBoost model with a distinct hyperparameter configuration sampled by the TPE algorithm. The best trial was identified by maximum mean CV AUC-ROC.

[Insert Figure 26: Optuna Optimisation History — CV AUC-ROC per Trial (30 Trials)] Caption: Line plot showing the cross-validation AUC-ROC for each of the 30 Optuna trials, with the Day 9 baseline AUC shown as a red dashed horizontal reference line. The best trial is annotated.

[Insert Table 17: Optuna Best Hyperparameters — XGBoost Churn Model] Caption: Best hyperparameter configuration identified by Optuna after 30 trials, alongside the Day 9 baseline values for comparison.

Hyperparameter	Baseline (Day 9)	Optuna Best
n_estimators	200	[ADD ACTUAL]
max_depth	5	[ADD ACTUAL]
learning_rate	0.05	[ADD ACTUAL]
subsample	0.8	[ADD ACTUAL]
colsample_bytree	0.8	[ADD ACTUAL]
min_child_weight	1 (default)	[ADD ACTUAL]
gamma	0 (default)	[ADD ACTUAL]
reg_alpha	0 (default)	[ADD ACTUAL]
reg_lambda	1 (default)	[ADD ACTUAL]

14.4.4 Tuned Model Evaluation

The tuned model was retrained on the full SMOTE training set using the best Optuna parameters and evaluated on the same held-out test set used for the baseline.

[Insert Figure 27: ROC Curve Comparison — Baseline vs Optuna-Tuned XGBoost] Caption: Side-by-side ROC curves for the baseline (Day 9) and Optuna-tuned (Day 11) XGBoost models on the identical test set, with AUC values annotated in the legend.

[Insert Figure 28: Confusion Matrix — Optuna-Tuned XGBoost (Day 11)] Caption: Confusion matrix for the tuned XGBoost model, showing improved true positive rate and reduced false negatives compared to the Day 9 baseline.

[Insert Table 18: Churn Model Performance — Baseline vs Optuna-Tuned] Caption: Side-by-side performance comparison of the baseline and tuned XGBoost models on the held-out test set, with project targets for reference.

Metric	Baseline (Day 9)	Tuned (Day 11)	Target
AUC-ROC	[ADD ACTUAL]	[ADD ACTUAL]	≥ 0.88
Precision@Top20%	[ADD ACTUAL]	[ADD ACTUAL]	≥ 0.75
Best CV AUC (Optuna)	—	[ADD ACTUAL]	—
AUC Improvement	—	[ADD ACTUAL]	Positive
Precision (class 1, threshold 0.5)	[ADD ACTUAL]	[ADD ACTUAL]	—
Recall (class 1, threshold 0.5)	[ADD ACTUAL]	[ADD ACTUAL]	—
F1-Score (class 1)	[ADD ACTUAL]	[ADD ACTUAL]	—

The tuned churn predictions for all customers — including ChurnProbability_Tuned and PredictedChurn_Tuned columns — were saved as [data/churn_predictions_tuned.csv](#) for use in the Streamlit dashboard and inventory analysis. The best hyperparameter set was persisted as [data/optuna_best_params.csv](#).

14.5 SHAP Explainability

14.5.1 Global Feature Importance

SHAP TreeExplainer was applied to the Optuna-tuned XGBoost model to quantify each feature's contribution to predictions across the test set. The TreeExplainer computes exact SHAP values for tree-based models in polynomial time, making it computationally feasible for the customer-level dataset.

```
explainer = shap.TreeExplainer(tuned_model)
shap_values = explainer.shap_values(X_test)
```

[Insert Figure 29: SHAP Summary Plot — Feature Impact on Churn Probability (Tuned Model)] Caption: SHAP beeswarm summary plot for all test-set customers. Each dot represents one customer; horizontal position shows the SHAP value (impact on churn probability); colour indicates feature value (red = high, blue = low). Features are ordered by mean absolute SHAP value.

[Insert Figure 30: SHAP Bar Plot — Mean Absolute Feature Importance (Tuned Model)] Caption: Mean absolute SHAP values per feature, providing a global ranking of feature importance for the tuned churn model. Recency is expected to rank first due to its role in defining the churn label.

14.5.2 Local Explanation — Highest-Risk Customer

A force plot was generated for the customer ranked highest by predicted churn probability, illustrating precisely which features pushed the prediction towards churn and which acted as protective factors.

[Insert Figure 31: SHAP Force Plot — Highest-Risk Customer in Test Set] Caption: SHAP force plot for the single customer with the highest predicted churn probability. Red arrows indicate features pushing the prediction towards churn (positive SHAP values); blue arrows indicate protective features. The baseline value (expected churn probability across all customers) and final prediction are annotated.

14.5.3 Explainability Insights

Across both global and local analyses, **Recency** consistently emerged as the dominant predictor, which is theoretically consistent with the churn label construction. However, the SHAP analysis also revealed that **Frequency** and **UniqueProducts** provided significant independent signal: customers with low Frequency and low UniqueProducts exhibited substantially higher SHAP contributions towards churn, even after controlling for Recency. This suggests that product engagement breadth is an independent early warning signal of disengagement — a finding with direct implications for product recommendation strategies.

15. Inventory Optimisation

Inventory optimisation was implemented in [Day10_InventoryOptimization.ipynb](#) using the 30-day ensemble demand forecast generated in Day 8 ([data/ensemble_future_30_days.csv](#)) as the primary input.

15.1 Theoretical Framework

The optimisation model is grounded in classical inventory management theory, specifically the **Continuous Review (Q, R) System**, which defines an order of fixed size Q to be placed whenever inventory falls to the reorder point R. The key parameters are computed as follows:

```
Safety Stock = Z × σ_demand × √(Lead Time)
Reorder Point = (μ_demand × Lead Time) + Safety Stock
Order Qty = max(0, Demand_30d + Safety Stock - Current Stock)
```

Where:

- **Z** = service level Z-score (95% service level → Z = 1.65)
- **σ_demand** = standard deviation of historical daily sales
- **μ_demand** = mean historical daily sales
- **Lead Time** = days from placing to receiving an order (default: 7 days)

15.2 Parameter Computation

Demand statistics were computed from `data/daily_sales.csv` (the historical daily sales series from Week 1):

[Insert Table 19: Inventory Optimisation — Computed Parameters] Caption: Inventory parameters computed from historical demand statistics and the 30-day ensemble forecast, using default settings of 7-day lead time and 95% service level.

Parameter	Formula / Source	Value
Average Daily Demand (μ)	Mean of <code>daily_sales["Sales"]</code>	[ADD ACTUAL]
Std Dev Daily Demand (σ)	Std of <code>daily_sales["Sales"]</code>	[ADD ACTUAL]
Lead Time (days)	Configurable (default: 7)	7 days
Service Level Z-score	95% → Z = 1.65	1.65
Safety Stock	$Z \times \sigma \times \sqrt{\text{Lead Time}}$	[ADD ACTUAL VALUE]
Reorder Point (ROP)	$(\mu \times \text{Lead Time}) + \text{Safety Stock}$	[ADD ACTUAL VALUE]
Assumed Current Stock	$15 \times \mu$ (configurable)	[ADD ACTUAL]
Total 30-Day Forecast	Sum of <code>ensemble_yhat</code>	[ADD ACTUAL]
Recommended Order Qty	$\max(0, \text{Demand}_{30d} + \text{SS} - \text{Stock})$	[ADD ACTUAL VALUE]
Overstock Ceiling	$2 \times \text{Reorder Point}$	[ADD ACTUAL]

15.3 30-Day Stock Projection

Starting from the assumed current stock level, daily ensemble demand forecasts were subtracted sequentially to simulate stock depletion over the 30-day planning horizon. Each day was then classified into one of three status categories:

- **Understock Risk:** Projected stock < Reorder Point
- **Optimal:** Reorder Point ≤ Projected stock ≤ Overstock Ceiling
- **Overstock Risk:** Projected stock > Overstock Ceiling

[Insert Figure 32: 30-Day Projected Stock vs Reorder Point and Safety Stock Reference Lines] Caption: Line plot of projected daily stock levels over the 30-day ensemble forecast window, with horizontal dashed lines for Reorder Point (red), Safety Stock (orange), and Overstock Ceiling (amber). Days in the Understock Risk zone are shaded red.

[Insert Figure 33: Stock Status Classification — Bar Chart coloured by Status] Caption: Daily bar chart of projected stock levels over 30 days, colour-coded by status (green = Optimal, red = Understock Risk, orange = Overstock Risk), providing an at-a-glance inventory health overview.

15.4 Reorder Alert Logic

A conditional reorder alert was implemented in both the notebook and the Streamlit dashboard:

```
below_rop = projection[projection["BelowReorderPoint"]]
if len(below_rop) > 0:
```

```
trigger_date = below_rop.iloc[0]["ds"]
days_until_reorder = (trigger_date - projection["ds"].iloc[0]).days + 1
```

If stock was projected to fall below the Reorder Point within the 30-day window, an alert was generated specifying the trigger date, days until reorder, and the recommended order quantity. If stock remained above the Reorder Point throughout the window, a confirmation of adequate stock was provided.

The 30-day projection and inventory summary were saved as `data/inventory_projection.csv` and `data/inventory_summary.csv` respectively, for consumption by the Streamlit Inventory Optimisation page.

15.5 Dashboard Integration

The inventory parameters in the Streamlit dashboard (Page 3) are fully interactive — the lead time, service level, and current stock assumptions can be adjusted via sidebar controls, with all charts and KPI metrics recalculating in real time without requiring a notebook re-run. This demonstrates the platform's capability to support operational what-if analysis.

16. Drift Detection and MLOps Pipeline

MLOps capabilities were implemented across Days 12 and 13, addressing the model degradation problem identified in the Problem Statement. The implementation covers two distinct components: statistical drift detection using Evidently AI (Day 12) and an automated, conditionally-branching retraining workflow using Apache Airflow (Day 13).

16.1 Evidently AI Drift Detection (Day 12)

16.1.1 Framework and API

Drift detection was implemented in `Day12_DriftDetection.ipynb` using **Evidently AI version 0.7.21**, which introduced a completely redesigned API compared to the widely documented 0.4.x series. The new API uses `Report`, `Dataset`, and `DataDefinition` objects, replacing the legacy `Dashboard` and `ProfileSection` classes:

```
from evidently import Report, Dataset, DataDefinition
from evidently.presets import DataDriftPreset, DataSummaryPreset

data_definition = DataDefinition()
reference_dataset = Dataset.from_pandas(reference_data, data_definition=data_definition)
current_dataset = Dataset.from_pandas(current_data, data_definition=data_definition)

drift_report = Report(metrics=[DataDriftPreset()])
result = drift_report.run(current_dataset, reference_dataset)
result.save_html("../reports/data_drift_report.html")
```

16.1.2 Reference and Current Dataset Construction

Since no real future data batch was available during the internship period, a production-representative drift scenario was simulated. The customer feature dataset (`churn_predictions_tuned.csv`) was divided into two halves by customer index:

- **Reference dataset:** First 50% of customers (represents the training-period data distribution)
- **Current dataset:** Second 50% of customers (represents a new incoming batch)

A synthetic drift was applied to three features in the current batch to simulate the effect of seasonal shifts in customer behaviour:

```
current_data["Recency"] *= rng.uniform(1.15, 1.35, len(current_data))
current_data["Monetary"] *= rng.uniform(0.75, 0.95, len(current_data))
current_data["AvgOrderValue"] *= rng.uniform(0.80, 1.00, len(current_data))
```

This simulates a scenario where customers are purchasing less recently and spending less — typical of a post-peak seasonal period.

16.1.3 Drift Test — Kolmogorov-Smirnov Test

Evidently AI applied the **Kolmogorov-Smirnov (K-S) test** to each numerical feature, comparing the empirical cumulative distribution functions (ECDFs) of the reference and current batches. A feature was flagged as drifted if the K-S test p-value fell below the significance threshold of **p < 0.05**.

[Insert Figure 34: Feature Drift Detection — K-S Test p-values (Bar Chart)] Caption: Horizontal bar chart of K-S test p-values for all eight monitored features. Bars to the left of the vertical dashed line at p=0.05 are coloured red (drifted); bars to the right are green (stable). Features with applied synthetic drift (Recency, Monetary, AvgOrderValue) are expected to register as drifted.

[Insert Figure 35: Drift Share Donut Chart — Drifted vs Stable Features] Caption: Donut chart showing the proportion of monitored features classified as drifted (red) vs stable (green), providing an overall drift health indicator.

16.1.4 Retraining Decision Rule

An overall drift share was computed as the fraction of monitored features with p < 0.05. A binary retraining decision was applied using a configurable threshold:

```
RETRAIN_THRESHOLD = 0.5
retrain_needed = drift_share >= RETRAIN_THRESHOLD
```

[Insert Table 20: Drift Detection Results Summary] Caption: Summary of the drift detection run, including the number of drifted features, overall drift share, applied threshold, and the resulting retraining recommendation.

Metric	Value
Total Features Monitored	8 (7 RFM/behavioural + ChurnProbability_Tuned)
Features Flagged as Drifted	[ADD ACTUAL]
Drift Share	[ADD ACTUAL VALUE]
Retrain Threshold	0.50 (50%)
Retraining Recommended	[ADD ACTUAL: True / False]

A full HTML drift report ([reports/data_drift_report.html](#)) and a data summary comparison report ([reports/data_summary_report.html](#)) were generated by Evidently AI and saved to the project's reports directory for documentation and stakeholder review. Structured CSV summaries ([data/drift_column_results.csv](#), [data/drift_monitor_summary.csv](#)) were also saved for consumption by the Streamlit Metrics & Alerts page.

16.2 Apache Airflow Retraining Workflow (Day 13)

16.2.1 Architecture

The automated retraining pipeline was implemented in [Day13_RetrainingPipeline.ipynb](#) as four composable Python functions in [pipeline_tasks.py](#), which were then wired into an Apache Airflow Directed Acyclic Graph (DAG) defined in [retraining_dag.py](#).

Since Apache Airflow requires a separate scheduler and webserver infrastructure that is impractical in a notebook-only environment, the pipeline functions were fully validated in the notebook through an end-to-end local simulation before being exported to the Airflow DAG file. This approach mirrors industry practice, where pipeline logic is developed and tested locally before being deployed to an Airflow cluster.

16.2.2 Pipeline Functions

[Insert Table 21: Retraining Pipeline Task Functions] Caption: The four Python functions in pipeline_tasks.py, with their signatures, responsibilities, and return values.

Function	Responsibility	Key Libraries	Return Value
<code>check_drift()</code>	Loads data, constructs reference/current batches, runs Evidently AI drift report, returns retrain flag	Evidently AI, Pandas	<code>{"drift_share": float, "retrain_needed": bool}</code>
<code>retrain_model()</code>	Loads latest data, applies SMOTE, trains XGBoost, evaluates on test split, saves model	XGBoost, imbalanced-learn, Scikit-learn	<code>{"auc": float, "model_path": str}</code>

Function	Responsibility	Key Libraries	Return Value
<code>evaluate_and_promote()</code>	Compares new model AUC against minimum threshold (0.80); copies retrained model to production path if passing	shutil, os	<code>{"auc": float, "passed_threshold": bool, "promoted": bool}</code>
<code>log_pipeline_run()</code>	Appends run metadata (timestamp, drift share, AUC, promotion status) to retraining_log.csv	Pandas, datetime	<code>log_entry: dict</code>

16.2.3 Airflow DAG Structure

The DAG (`retailpulse_churn_retraining`) is scheduled to run daily at midnight and uses a `BranchPythonOperator` to implement conditional retraining:

```
with DAG(
    dag_id="retailpulse_churn_retraining",
    schedule="@daily",
    start_date=datetime(2026, 1, 1),
    catchup=False,
    tags=["retailpulse", "mlops", "churn"]
) as dag:

    check_drift_task = BranchPythonOperator(task_id="check_drift_task", ...)
    retrain_task     = PythonOperator(task_id="retrain_task", ...)
    skip_retrain_task = EmptyOperator(task_id="skip_retrain_task")
    evaluate_task     = PythonOperator(task_id="evaluate_task", ...)
    log_task          = PythonOperator(task_id="log_task",
                                      trigger_rule="none_failed_min_one_success")

    check_drift_task >> [retrain_task, skip_retrain_task]
    retrain_task >> evaluate_task >> log_task
    skip_retrain_task >> log_task
```

The `trigger_rule="none_failed_min_one_success"` on `log_task` ensures that pipeline run metadata is always persisted to `retraining_log.csv`, regardless of whether the retrain branch or skip branch was taken.

[Insert Figure 36: Apache Airflow DAG Architecture Diagram] Caption: Visual representation of the Airflow DAG showing the `BranchPythonOperator` fan-out from `check_drift_task` to `retrain_task` and `skip_retrain_task`, converging at `evaluate_task` and `log_task`. Conditional arrows are labelled with their branching conditions.

16.2.4 Model Promotion Logic

The `evaluate_and_promote` function implements a minimum quality gate before promoting a newly retrained model to production:

```
MIN_AUC_THRESHOLD = 0.80

if auc >= MIN_AUC_THRESHOLD and promote:
    shutil.copy(retrain_result["model_path"], MODEL_PATH)
    promoted = True
```

If the retrained model fails to meet the AUC threshold of 0.80, it is saved as a candidate model (`models/xgb_churn_model_retrained.json`) but the production model (`models/xgb_churn_model_tuned.json`) remains unchanged. This prevents model regression due to degraded retraining data quality.

16.2.5 Audit Logging

Every pipeline execution appends a structured record to `data/retraining_log.csv`, creating an auditable history of all drift check results, retraining decisions, new model performance, and promotion outcomes. This log is displayed directly in the Streamlit Metrics & Alerts dashboard page.

17. Streamlit Dashboard

The interactive analytics dashboard was developed across Days 15–20 using **Streamlit 1.35+** as the web framework and **Plotly 5.20+** for all interactive charts. The dashboard comprises a Home page ([app.py](#)) and five dedicated analytical pages, all sharing a centralised data loading utility ([utils/data_loader.py](#)).

Architecture Decision — Centralised Data Loader

All 19 CSV artefacts from Weeks 1 and 2 are loaded through a single `load_all_data()` function that uses `os.path.abspath` to resolve the `data/` directory relative to the dashboard's installation location, regardless of the working directory from which Streamlit is launched:

```
_UTILS_DIR      = os.path.dirname(os.path.abspath(__file__)) # utils/
_DASHBOARD_DIR  = os.path.dirname(_UTILS_DIR)               # retailpulse_dashboard/
_ROOT_DIR       = os.path.dirname(_DASHBOARD_DIR)           # RetailPulse/ (project root)
DATA_DIR        = os.path.join(_ROOT_DIR, "data")
```

This design ensures that the dashboard is portable across different operating systems and working directories, and that all pages share a single, consistent data access layer with graceful `None` returns for any missing files.

[Insert Figure 37: RetailPulse Dashboard Home Page Screenshot] Caption: Home page ([app.py](#)) showing the dark-background hero banner, four live KPI metric cards (Forecast MAPE, Churn AUC-ROC, At-Risk Customers, Reorder Point), module overview cards, and Week 2 Targets Summary table. KPIs populate automatically when Week 2 CSV artefacts are present.

17.1 Demand Forecasting Page (Day 16)

[pages/1_Demand_Forecasting.py](#) provides a comprehensive forecasting analytics interface with four interactive sections and a sidebar containing date range and visualisation filters.

Sidebar Controls:

- Date range selector (Last 90 days / Last 180 days / All data) for the historical chart
- Prophet confidence interval toggle (show/hide 95% prediction interval shading)

Section 1 — Historical vs Ensemble Forecast

An interactive Plotly line chart overlays actual daily sales (blue), the ensemble forecast (red dashed), and optionally the Prophet 95% confidence interval (shaded). The `hovermode="x_unified"` setting enables simultaneous tooltip display for all traces at a given date.

Section 2 — 30-Day Future Demand Forecast

All three models (Prophet, LSTM, Ensemble) are plotted together on the forward-looking 30-day window, with the forecast region highlighted by a vertical rectangle annotation. Two KPI cards display the total 30-day forecasted demand and the peak forecast day.

Section 3 — Model Comparison

A 60-day window comparison chart overlays actual sales against all three model predictions, enabling direct visual assessment of the ensemble's improvement over its components. A companion bar chart compares MAPE values across the three models with a target reference line at 12%.

Section 4 — What-If Analysis

An interactive demand multiplier slider ($0.5\times$ to $2.0\times$) allows users to simulate business scenarios such as promotions (multiplier > 1.0), supply shocks, or economic downturns (multiplier < 1.0). The adjusted forecast, suggested order quantity (adjusted demand + 10% buffer), and a fill-between area chart showing baseline vs adjusted demand update in real time.

[Insert Figure 38: Demand Forecasting Page — What-If Analysis Slider and Adjusted Forecast Chart] Caption: What-if analysis panel showing the demand multiplier slider set to $1.3\times$ (surge scenario), with adjusted 30-day total demand, suggested order quantity, and the baseline vs adjusted demand chart with shaded difference area.

17.2 Customer Segmentation and Churn Risk Page (Day 17)

`pages/2_Customer_Segmentation.py` provides a four-section customer analytics interface with a sidebar churn threshold slider (0.30 to 0.90, default 0.50) and a top-N customer selector (10, 20, 50, 100).

Section 1 — RFM Segment Distribution

A Plotly bar chart and companion donut chart display customer counts by RFM segment. A scatter plot (Recency vs Frequency, bubble size = Monetary) and a box plot (Monetary by segment) provide multidimensional segment profiling.

Section 2 — Churn Risk Score Distribution

An overlapping histogram displays the churn probability distribution split into Active (green) and At-Risk (red) populations, with a vertical threshold line adjusting dynamically with the sidebar slider. A companion chart shows average churn risk per RFM segment, enabling segment-level retention prioritisation.

Section 3 — Top-N At-Risk Customers

A colour-coded data table (red > 0.8, amber > 0.6, yellow > 0.5) ranks customers by `ChurnProbability_Tuned`, displaying their Recency, Frequency, Monetary, and segment label. A customer risk map scatter plot (Recency vs Monetary, colour-coded by churn probability) provides a spatial view of the customer population's risk landscape.

Section 4 — Model Performance and Feature Importance

The Optuna tuning summary and churn metrics tables are displayed alongside a bar chart of feature-churn probability correlations, providing a proxy feature importance view accessible without re-running the SHAP analysis.

[Insert Figure 39: Customer Segmentation Page — Customer Risk Map Scatter Plot] Caption: Scatter plot of all customers in Recency–Monetary space, colour-coded by churn probability (red = high risk, green = low risk), with the threshold-based at-risk count and churn rate displayed as KPI cards above.

17.3 Inventory Optimisation Page (Day 18)

`pages/3_Inventory_Optimization.py` is notable for its **live parameter recalculation** capability: all inventory metrics (safety stock, reorder point, order quantity, projected stock levels) are recomputed in real time from the ensemble forecast whenever the user adjusts any sidebar control.

Sidebar Controls:

- Lead time slider (1–30 days)
- Service level dropdown (90% / 95% / 98% / 99%)
- Current stock slider (5–60× average daily demand)

Section 1 — Parameters and Stock Health Gauge

A full parameter table displays all computed inventory values. A Plotly gauge chart visualises current stock as a percentage of the reorder point, with colour zones indicating healthy (green, 100–150%), understocked (red, <100%), and overstocked (amber, >150%) regions.

Section 2 — 30-Day Stock Projection

The stock projection chart plots daily projected stock levels with three reference lines (Reorder Point, Safety Stock, Overstock Ceiling) and a shaded understock danger zone. Points are colour-coded by their daily status classification. A conditional alert banner (red error or green success) below the chart indicates whether a reorder is required within the 30-day window, with the exact trigger date and recommended order quantity.

Section 3 — Status Classification and Section 4 — Recommendations

A donut chart summarises how many of the 30 forecasted days fall into each status category. A daily forecasted demand bar chart provides context for understanding the stock depletion rate.

[Insert Figure 40: Inventory Optimisation Page — 30-Day Stock Projection with Reorder Alert] Caption: Stock projection line chart showing projected stock depletion against the reorder point (red dashed), safety stock (orange dotted), and overstock ceiling (amber dashed). A red error banner below the chart indicates the date on which stock is projected to fall below the reorder point, with the recommended order quantity displayed.

17.4 Metrics and Alerts Page (Day 19)

`pages/4_Metrics_and_Alerts.py` serves as the operational monitoring hub for the RetailPulse platform, aggregating model performance, drift status, and pipeline health into a single view.

Live Alert System

At the top of the page, conditional alert banners are computed dynamically from sidebar threshold sliders. If the Ensemble MAPE exceeds the user-defined MAPE threshold, or the Churn AUC-ROC falls below the AUC threshold, or the drift share exceeds the drift threshold, a red error banner is displayed with the offending metric value. If all metrics are within bounds, a single green success banner confirms system health.

Sidebar Controls:

- MAPE alert threshold slider (0.05 to 0.30)
- AUC-ROC alert threshold slider (0.70 to 0.95)
- Drift share alert threshold slider (0.10 to 0.90)

Section 1 — Model Performance vs Targets

Three columns display forecasting KPIs (MAPE and RMSE per model with target met/failed indicators), churn model KPIs (AUC-ROC, Precision@Top20%, Best CV AUC, AUC improvement), and the Week 2 targets summary table with green/red row-level highlighting.

Radar Chart

A Plotly radar chart overlays actual performance values against project targets across all key metrics simultaneously, providing a compact multi-dimensional performance view. The gap between the actual and target polygons immediately communicates which dimensions require improvement.

[Insert Figure 41: Metrics and Alerts Page — Radar Chart and Live Alert Banners] Caption: Top section showing the alert banner system (green success in this case) and the radar chart comparing actual model performance (blue filled polygon) against targets (red dashed polygon) across Forecast Accuracy (1-MAPE), Churn AUC-ROC, Precision@Top20%, and CV AUC dimensions.

Section 2 — Data Drift Status

The K-S test p-value bar chart from `drift_column_results.csv` is displayed with colour-coded bars (red = drifted, green = stable), the drift share donut chart, the drift summary table, and a conditional retraining recommendation banner. A reference to the full HTML Evidently AI report (`reports/data_drift_report.html`) is provided.

Section 3 — Retraining Pipeline Log

The `retraining_log.csv` is displayed as a colour-coded table (green = promoted, amber = retrained but not promoted, grey = skipped). If multiple pipeline runs have been logged, an AUC-over-time line chart tracks the retrained model quality across successive runs.

17.5 Export Reports Page (Day 20)

`pages/5_Export_Reports.py` delivers the reporting and export functionality specified in functional requirement F-07.

CSV Download Section

13 individual `st.download_button` components provide one-click download access to all Week 1 and Week 2 data artefacts, organised in three columns. Each button is rendered as enabled (with download functionality) if the corresponding CSV exists in `data/`, or as a disabled greyed-out button with a tooltip directing the user to the corresponding notebook if the file is absent. This provides clear guidance on execution dependencies without displaying error messages.

[Insert Table 22: Available CSV Downloads in the Export Reports Page] Caption: Complete list of 13 CSV files available for download through the Export Reports page, with their source notebooks and data content descriptions.

#	File	Source	Content
1	<code>ensemble_forecast_results.csv</code>	Day 8	Historical + ensemble predictions with actuals
2	<code>ensemble_future_30_days.csv</code>	Day 8	30-day future forecast (Prophet, LSTM, Ensemble)
3	<code>ensemble_metrics.csv</code>	Day 8	MAPE and RMSE per model
4	<code>churn_predictions_tuned.csv</code>	Day 11	Customer-level churn probabilities (tuned model)

#	File	Source	Content
5	churn_metrics.csv	Day 9	AUC-ROC and Precision@Top20%
6	optuna_tuning_summary.csv	Day 11	All tuning metrics and improvement delta
7	optuna_best_params.csv	Day 11	Best hyperparameter configuration
8	inventory_projection.csv	Day 10	30-day stock status per day
9	inventory_summary.csv	Day 10	Safety stock, ROP, order quantity parameters
10	drift_column_results.csv	Day 12	Per-feature K-S p-values and drift flags
11	drift_monitor_summary.csv	Day 12	Overall drift share and retrain decision
12	retraining_log.csv	Day 13	Auditable pipeline run history
13	week2_targets_summary.csv	Day 14	Project targets vs achieved metrics

HTML/PDF Report Generator

A `build_pdf_html()` function dynamically constructs a styled HTML document incorporating data from whichever sections the user selects via sidebar checkboxes (Demand Forecasting, Churn Prediction, Inventory Optimisation, Drift & MLOps). The HTML report includes:

- A styled cover section with report timestamp and project metadata
- KPI summary cards for each selected module
- Rendered data tables (top 20 at-risk customers, model metrics, inventory parameters, drift results)
- Conditional alert blocks (green/red) for reorder status and retraining recommendations
- A project technology stack table and footer

The report is downloaded as an `.html` file with a timestamped filename. Users can open it in any modern browser and use the browser's print function (Ctrl+P → Save as PDF) to produce a clean, styled PDF document — without requiring any server-side PDF generation dependencies.

[Insert Figure 42: Export Reports Page — CSV Download Grid and HTML Report Generator] Caption: Export Reports page showing the 3-column grid of 13 CSV download buttons (enabled buttons shown in blue, disabled in grey) and the HTML report section with module checkboxes, report preview description, and the Download Report button.

18. Results and Evaluation Metrics

18.1 Demand Forecasting Results

The ensemble forecasting model achieved a MAPE of **[ADD ACTUAL VALUE]** on the validation period, against the project target of $\leq 12\%$. The optimal blending weight was **[ADD ACTUAL VALUE]** for Prophet and **[ADD ACTUAL VALUE]** for LSTM, indicating that [Prophet / LSTM] contributed more strongly to the ensemble's accuracy on this dataset's temporal patterns.

[Insert Table 23: Comprehensive Forecasting Results] Caption: MAPE and RMSE values for all three forecasting models on the held-out validation period, with target compliance indicators.

Model	MAPE	RMSE (£)	Target (MAPE $\leq 12\%$)	Status
Prophet	[ADD ACTUAL VALUE]	[ADD ACTUAL VALUE]	—	—
LSTM	[ADD ACTUAL VALUE]	[ADD ACTUAL VALUE]	—	—
Ensemble	[ADD ACTUAL VALUE]	[ADD ACTUAL VALUE]	$\leq 12\%$	[Met / Not Met]

18.2 Churn Prediction Results

The Optuna-tuned XGBoost model achieved an AUC-ROC of **[ADD ACTUAL VALUE]** (target ≥ 0.88) and a Precision@Top20% of **[ADD ACTUAL VALUE]** (target ≥ 0.75). Optuna's 30-trial search improved the baseline AUC-ROC by **[ADD ACTUAL VALUE]** percentage points, demonstrating the value of systematic hyperparameter optimisation over default configurations.

[Insert Table 24: Comprehensive Churn Model Results] Caption: Full performance comparison across baseline (Day 9) and tuned (Day 11) XGBoost churn models, with project targets and compliance indicators.

Metric	Baseline (Day 9)	Tuned (Day 11)	Target	Status
AUC-ROC	[ADD ACTUAL]	[ADD ACTUAL VALUE]	≥ 0.88	[Met / Not Met]
Precision@Top20%	[ADD ACTUAL]	[ADD ACTUAL VALUE]	≥ 0.75	[Met / Not Met]
Best CV AUC (Optuna)	—	[ADD ACTUAL VALUE]	—	—
AUC Improvement	—	[ADD ACTUAL VALUE]	Positive	✓
Precision (Churned class)	[ADD ACTUAL]	[ADD ACTUAL]	—	—
Recall (Churned class)	[ADD ACTUAL]	[ADD ACTUAL]	—	—
F1-Score (Churned class)	[ADD ACTUAL]	[ADD ACTUAL]	—	—

18.3 Inventory Optimisation Results

[Insert Table 25: Inventory Optimisation Results] Caption: Computed inventory parameters and 30-day stock status classification results.

Metric	Value
Safety Stock	[ADD ACTUAL VALUE] units
Reorder Point	[ADD ACTUAL VALUE] units
Recommended Order Quantity	[ADD ACTUAL VALUE] units
Days in Optimal Zone	[ADD ACTUAL] / 30
Days in Understock Risk Zone	[ADD ACTUAL] / 30
Days in Overstock Risk Zone	[ADD ACTUAL] / 30
Reorder Alert Triggered	[Yes / No]

18.4 Drift Detection Results

[Insert Table 26: Drift Detection and MLOps Pipeline Results] Caption: Summary of drift detection metrics and retraining pipeline outcomes from Day 12 and Day 13.

Metric	Value
Features Monitored	8
Features Drifted (p < 0.05)	[ADD ACTUAL]
Drift Share	[ADD ACTUAL VALUE]
Retraining Triggered	[Yes / No]
Retrained Model AUC	[ADD ACTUAL]
Model Promoted to Production	[Yes / No]

18.5 Dashboard Delivery

[Insert Table 27: Dashboard Delivery Summary] Caption: Summary of all dashboard pages delivered in Week 3, with interactive control counts and key visualisation types.

Page	Day	Interactive Controls	Chart Types
Home	15	—	KPI metrics, module cards
Demand Forecasting	16	3 (date range, CI toggle, demand slider)	Line, bar, scatter, fill-between
Segmentation & Churn	17	2 (threshold slider, top-N selector)	Bar, pie, scatter, box, histogram
Inventory Optimisation	18	3 (lead time, service level, stock slider)	Line, bar, pie, gauge
Metrics & Alerts	19	3 (MAPE, AUC, drift threshold sliders)	Radar, bar, donut, line, table
Export Reports	20	17 (4 section checkboxes, 13 download buttons)	—

19. Challenges Faced

19.1 Evidently AI 0.7 API Migration

The project specification and available online documentation referenced Evidently AI's legacy 0.4.x API, which uses `Dashboard`, `ProfileSection`, and `ColumnDriftMetric` classes. Evidently AI 0.7 introduced a completely redesigned API with `Report`, `Dataset`, and `DataDefinition` as the primary abstractions, rendering all 0.4.x code incompatible. This required reading the official source code, experimenting with the new method signatures, and developing custom result-parsing logic to extract per-feature drift p-values from the new `result.dict()` output structure. The challenge was resolved by installing the latest Evidently AI version (0.7.21), reading the library's own test suite, and iteratively testing the API against synthetic data before applying it to the project data.

19.2 Double-Nested Folder Path Resolution in Streamlit

The project's actual folder structure on the development machine (`C:\Users\khush\RetailPulse\RetailPulse\`) contains a double `RetailPulse` nesting that differs from the assumed single-level structure in the initial `data_loader.py` implementation. The initial relative path (`../data/`) from within `retailpulse_dashboard/utils/` resolved to the outer `RetailPulse/data/` rather than the correct inner `RetailPulse/data/`. The fix required using `os.path.abspath(__file__)` to establish the absolute path of `data_loader.py` at runtime and walking up exactly two directory levels to reach the correct project root, making the solution robust to any nesting depth.

19.3 Streamlit Invocation Error

The Streamlit application was initially invoked using `python app.py`, which triggered the Streamlit runtime in "bare mode" — producing a flood of `missing ScriptRunContext` warnings and no browser window. Streamlit is a specialised web framework that must be launched through its own CLI (`streamlit run app.py`), which initialises the server, event loop, and session context required for the reactive component model. This distinction between Python script execution and Streamlit server execution is not immediately obvious from the script file itself.

19.4 Recursive LSTM Forecasting Error Accumulation

Multi-step recursive forecasting with the LSTM model — where each prediction is fed back as input for the next step — introduces compounding prediction errors over the 30-day horizon. Errors that are small in the first one or two steps become progressively amplified in later steps, causing the 30-day LSTM forecast to drift from the expected seasonal range. The ensemble's weighting mechanism partially mitigated this by combining the LSTM's short-term pattern accuracy with Prophet's more stable long-range trend and seasonality extrapolation.

19.5 SMOTE Data Leakage Prevention

An early implementation risk involved applying SMOTE before the train/test split, which would generate synthetic samples that statistically influence the test set, invalidating the evaluation. The correct implementation strictly applies SMOTE only to `X_train` after splitting, using a pipeline-first approach. This was identified and corrected during a code review step, and the final implementation explicitly documents the SMOTE placement rationale in notebook cell comments.

19.6 Airflow Infrastructure Constraints

Apache Airflow's production setup requires a running scheduler process, a metadata database (PostgreSQL or SQLite), a webserver, and appropriate Python environment configuration — all of which are impractical in a notebook-only internship environment. The solution was to implement and fully validate all pipeline logic as pure Python functions (verifiable in a standard Jupyter kernel), generate the Airflow DAG file as a deployment-ready artefact, and document the production deployment steps in the README. This approach is consistent with real-world MLOps practice, where DAG code is developed locally and deployed to a managed Airflow cluster.

20. Business Impact

RetailPulse directly addresses four commercially significant retail challenges. The quantified business impact targets set by Zidio Development, alongside the capabilities implemented to achieve them, are summarised below.

[Insert Table 28: Business Impact Summary] Caption: Mapping of RetailPulse capabilities to quantified business impact targets and the specific technical implementation delivering each impact.

Challenge	Capability Delivered	Quantified Impact Target	Implementation
Demand Uncertainty	Hybrid Prophet+LSTM ensemble, 30-day forward forecast	30–50% reduction in stockouts	<code>ensemble_future_30_days.csv</code> , Forecasting dashboard page

Challenge	Capability Delivered	Quantified Impact Target	Implementation
Customer Attrition	XGBoost churn model, Precision@Top20% ≥ 0.75	Identify and target top 20% of at-risk customers with 75%+ precision	<code>churn_predictions_tuned.csv</code> , Segmentation dashboard page
Inventory Inefficiency	Demand-driven safety stock and reorder point	25–40% reduction in overstock/understock	<code>inventory_projection.csv</code> , Inventory dashboard page
Revenue Opportunity	Better stock availability through accurate forecasting	15–25% revenue increase	Ensemble forecast + inventory integration
Silent Model Degradation	Evidently AI drift detection + Airflow retraining	Automated daily monitoring, proactive retraining	<code>retraining_dag.py</code> , Metrics & Alerts page
Analytical Accessibility	5-page interactive Streamlit dashboard	Democratised ML insights for non-technical stakeholders	<code>retailpulse_dashboard/</code>
Scalability	Daily batch architecture designed for 10M+ transactions	<5 min daily batch processing	Pandas-based ETL, CSV artefact pipeline

The Precision@Top20% metric of **[ADD ACTUAL VALUE]** has a direct and quantifiable commercial interpretation: in a retention campaign targeting the top 20% of customers by churn risk, **[ADD ACTUAL VALUE]** of contacted customers would genuinely be at risk of churning. Assuming a customer lifetime value (CLV) of £500 per retained customer and a campaign cost of £20 per contact, a Precision@Top20% of 0.75 means that 75% of campaign spend delivers genuine retention value, compared to approximately 25% for random outreach — a **3× improvement in campaign efficiency**.

Similarly, the inventory optimisation module's ability to project stock depletion 30 days ahead, with configurable service levels and lead times, enables procurement teams to place orders with sufficient advance notice to avoid both stockouts (lost sales) and emergency procurement premiums (cost overruns).

21. Future Enhancements

21.1 Week 4 — Deployment and Production Polish (Days 22–28)

The following enhancements are planned within the project's 28-day execution plan but fall outside the scope of this report:

- **Docker Containerisation (Day 22):** Multi-stage Docker builds will package the Streamlit dashboard, pipeline tasks, and all dependencies into a portable container image, eliminating environment setup friction for reviewers and deployment targets. A `Dockerfile` and `docker-compose.yml` will be provided.
- **Kubernetes Deployment (Day 23):** Kubernetes manifests (Deployment, Service, ConfigMap, Horizontal Pod Autoscaler) will enable scalable, fault-tolerant production deployment of the dashboard with resource limits and health probes.
- **GitHub Actions CI/CD Pipeline (Day 24):** A workflow file (`.github/workflows/deploy.yml`) will automate linting, unit test execution, Docker image build and push to a container registry, and Streamlit Community Cloud or AWS deployment on every push to the main branch.
- **Cloud Deployment (Day 25):** The dashboard will be deployed to a publicly accessible HTTPS URL using Streamlit Community Cloud (free tier) or AWS Elastic Container Service, satisfying the live demo requirement of the Zidio Development submission guidelines.
- **Prometheus and Grafana Monitoring (Day 26):** Application-level metrics (prediction latency, data freshness, model output distribution) will be exposed via a Prometheus `/metrics` endpoint and visualised in Grafana dashboards.
- **Load Testing and Accuracy Validation (Day 27):** Locust-based load testing will validate that the dashboard meets the <5-minute daily batch processing requirement under concurrent user load.

21.2 Longer-Term Research Extensions

Beyond the 28-day project scope, several technically rich extensions would significantly enhance the platform's commercial value:

- **SKU-Level Granular Forecasting:** The current platform forecasts aggregate daily sales across all products. Per-StockCode forecasting would enable individual product-level inventory management, requiring hierarchical forecasting approaches (e.g., temporal hierarchical reconciliation or DeepAR).

- **Real-Time Streaming Integration:** Replacing the batch CSV pipeline with an Apache Kafka + Faust (or Apache Flink) streaming architecture would enable near-real-time demand signals and churn probability updates as transactions occur.
- **External Feature Integration:** Incorporating exogenous features — UK public holiday calendar, macroeconomic indicators (CPI, consumer confidence), competitor promotional calendars, and weather data — into the Prophet model via the `add_regressor()` API would improve forecast accuracy during anomalous periods.
- **Multi-Channel Attribution:** The dataset contains transactions from both domestic (UK) and international customers. Separate forecasting and churn models per major country or region could improve accuracy for internationally diverse retailer portfolios.
- **Natural Language Query Interface:** Integrating a large language model (LLM) as a conversational query layer on top of the Streamlit dashboard would enable non-technical business users to ask questions such as "Which customer segments have the highest churn risk this month?" in plain English, with the LLM translating the query to dashboard filter states or data aggregations.
- **Causal Inference for Churn:** Replacing purely predictive churn modelling with a causal uplift model (e.g., S-learner or T-learner meta-learner) would identify not just which customers are likely to churn, but which customers would respond most positively to retention interventions — enabling truly incremental campaign targeting.

22. Conclusion

This internship project successfully delivered **RetailPulse**, an end-to-end AI-powered retail analytics platform, across three completed weeks of a structured 28-day programme at Zidio Development. The platform comprehensively addresses four core retail business challenges — demand uncertainty, customer attrition, inventory inefficiency, and silent model degradation — through an integrated stack of machine learning, deep learning, and MLOps capabilities.

The key technical contributions of the project are:

1. A **12-step data cleaning and feature engineering pipeline** that transformed the Online Retail II dataset from 8 raw columns to 27 enriched features, establishing a high-quality analytical foundation
2. An **RFM-based customer segmentation** system using K-Means ($k=6$, selected by elbow and silhouette analysis) with business-interpretable segment labels
3. A **hybrid Prophet+LSTM ensemble forecasting model** with an optimal weight grid search, achieving a MAPE of **[ADD ACTUAL VALUE]** against a $\leq 12\%$ target
4. An **Optuna-tuned XGBoost churn classifier** with SMOTE class balancing and SHAP explainability, achieving an AUC-ROC of **[ADD ACTUAL VALUE]** (target ≥ 0.88) and Precision@Top20% of **[ADD ACTUAL VALUE]** (target ≥ 0.75)
5. A **demand-driven inventory optimisation engine** with configurable safety stock, reorder point, and recommended order quantity calculations
6. An **Evidently AI drift detection pipeline** (K-S test, 0.7 API) integrated with an **Apache Airflow retraining DAG** implementing conditional retraining with a model quality gate
7. A **five-page interactive Streamlit dashboard** with real-time parameter controls, Plotly charts, and CSV/PDF export functionality

The project demonstrated proficiency across the full data science lifecycle and exposed several industry-relevant challenges — API version migration, path resolution in multi-directory projects, recursive forecasting error accumulation, and the operational constraints of ML pipeline orchestration — each of which was systematically diagnosed and resolved.

The internship experience at Zidio Development provided direct, hands-on exposure to the tools, workflows, and decision-making processes of professional data science practice, and has significantly strengthened the technical and methodological foundation required for a career in the field.

References

- [1] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Hoboken, NJ, USA: Wiley, 2015.
- [2] S. J. Taylor and B. Letham, "Forecasting at scale," *The American Statistician*, vol. 72, no. 1, pp. 37–45, Jan. 2018. doi: 10.1080/00031305.2017.1380080.
- [3] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997. doi: 10.1162/neco.1997.9.8.1735.
- [4] P. G. Zhang, "Time series forecasting using a hybrid ARIMA and neural network model," *Neurocomputing*, vol. 50, pp. 159–175, Jan. 2003. doi: 10.1016/S0925-2312(01)00702-0.

- [5] W. Verbeke, K. Dejaeger, D. Martens, J. Hur, and B. Baesens, "New insights into churn prediction in the telecommunication sector: A profit driven data mining approach," *European Journal of Operational Research*, vol. 218, no. 1, pp. 211–229, Apr. 2012. doi: 10.1016/j.ejor.2011.09.031.
- [6] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, USA, Aug. 2016, pp. 785–794. doi: 10.1145/2939672.2939785.
- [7] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, Jun. 2002. doi: 10.1613/jair.953.
- [8] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, Long Beach, CA, USA, Dec. 2017, pp. 4765–4774.
- [9] A. M. Hughes, *Strategic Database Marketing*, 4th ed. New York, NY, USA: McGraw-Hill, 2012.
- [10] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proc. 5th Berkeley Symp. on Mathematical Statistics and Probability*, Berkeley, CA, USA, 1967, vol. 1, pp. 281–297.
- [11] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. 2nd Int. Conf. Knowledge Discovery and Data Mining (KDD-96)*, Portland, OR, USA, Aug. 1996, pp. 226–231.
- [12] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems 28 (NeurIPS)*, Montreal, QC, Canada, Dec. 2015, pp. 2503–2511.
- [13] Evidently AI, "Evidently AI: Open-source ML monitoring and testing," 2021. [Online]. Available: <https://www.evidentlyai.com>. [Accessed: Jun. 2026].
- [14] A. Beauchemin, "Introducing Airflow: A workflow management platform," *Airbnb Engineering & Data Science Blog*, Jul. 2015. [Online]. Available: <https://medium.com/airbnb-engineering>. [Accessed: Jun. 2026].
- [15] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, Anchorage, AK, USA, Aug. 2019, pp. 2623–2631. doi: 10.1145/3292500.3330701.
- [16] D. Chen, S. L. Sain, and K. Guo, "Data mining for the online retail industry: A case study of RFM model-based customer segmentation using data mining," *Journal of Database Marketing & Customer Strategy Management*, vol. 19, no. 3, pp. 197–208, Sep. 2012. doi: 10.1057/dbm.2012.17.
- [17] Y. Koren, R. Bell, and C. Volinsky, "Matrix factorization techniques for recommender systems," *IEEE Computer*, vol. 42, no. 8, pp. 30–37, Aug. 2009. doi: 10.1109/MC.2009.263.
- [18] C. Chatfield, *The Analysis of Time Series: An Introduction with R*, 7th ed. Boca Raton, FL, USA: CRC Press / Chapman & Hall, 2019.
- [19] H. Abdi and L. J. Williams, "Principal component analysis," *WIREs Computational Statistics*, vol. 2, no. 4, pp. 433–459, Jul./Aug. 2010. doi: 10.1002/wics.101.
- [20] F. Pedregosa, G. Varoquaux, A. Gramfort *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, Nov. 2011.

Appendices

Appendix A: Notebook Execution Order and Artefact Dependencies

[Insert Table 29: Complete Notebook Execution Order with Input Dependencies and Output Artefacts] Caption: Sequential execution order for all 14 Week 1 and Week 2 notebooks, showing which artefacts each notebook depends on and produces.

Day	Notebook	Input Dependencies	Primary Output Artefacts
1	Day1_EDA.ipynb	online_retail_II.xlsx	EDA charts, statistical summaries
2	Day2_DataCleaning_FeatureEngineering.ipynb	online_retail_II.xlsx	cleaned_retail.csv (27 cols)
3	Day3_CustomerSegmentation.ipynb	cleaned_retail.csv	rfm_segmented.csv

Day	Notebook	Input Dependencies	Primary Output Artefacts
4	Day4_TimeSeriesPreparation.ipynb	cleaned_retail.csv	daily_sales.csv, time_series_prepared.csv
5	Day5_ProphetForecasting.ipynb	daily_sales.csv	forecast_results.csv, future_30_days_sales.csv
6	Day6_LSTMForecasting.ipynb	daily_sales.csv	lstm_predictions.csv, models/lstm_model.pth
7	Day7_Week1_Checkpoint.ipynb	All Week 1 CSVs	Week 1 summary visualisations
8	Day8_HybridEnsembleForecasting.ipynb	daily_sales.csv, forecast_results.csv	ensemble_forecast_results.csv, ensemble_future_30_days.csv, ensemble_metrics.csv, models/lstm_model_ensemble.pth
9	Day9_ChurnPrediction.ipynb	rfm_segmented.csv, cleaned_retail.csv	churn_predictions.csv, churn_metrics.csv, models/xgb_churn_model.json
10	Day10_InventoryOptimization.ipynb	daily_sales.csv, ensemble_future_30_days.csv	inventory_projection.csv, inventory_summary.csv
11	Day11_OptunaTuning.ipynb	churn_predictions.csv	churn_predictions_tuned.csv, optuna_tuning_summary.csv, optuna_best_params.csv, models/xgb_churn_model_tuned.json
12	Day12_DriftDetection.ipynb	churn_predictions_tuned.csv	drift_column_results.csv, drift_monitor_summary.csv, reports/data_drift_report.html
13	Day13_RetrainingPipeline.ipynb	All Week 2 CSVs	pipeline_tasks.py, retraining_dag.py, retraining_log.csv
14	Day14_Week2_Checkpoint.ipynb	All Week 2 CSVs	week2_targets_summary.csv

Appendix B: Dashboard Run Instructions

The following steps are required to launch the RetailPulse Streamlit dashboard locally:

Step 1 — Prerequisites

Ensure all 14 notebooks (Days 1–14) have been executed in order and all CSV artefacts are present in the `data/` directory. Refer to Appendix A for the execution order and dependency chain.

Step 2 — Install Dependencies

```
pip install streamlit plotly pandas numpy xgboost shap optuna imbalanced-learn evidently prophet torch
```

Or install from the requirements file:

```
pip install -r retailpulse_dashboard/requirements.txt
```

Step 3 — Navigate to Dashboard Directory

```
cd C:\Users\khush\RetailPulse\RetailPulse\retailpulse_dashboard
```

Step 4 — Launch the Dashboard

```
streamlit run app.py
```

Note: The application must be launched using `streamlit run app.py`, not `python app.py`. The latter runs the script in bare Python mode and produces `ScriptRunContext` warnings without opening a browser window.

Step 5 — Open Browser

The dashboard will automatically open at `http://localhost:8501`. All five pages are accessible from the left navigation pane.

Appendix C: Project Submission Checklist

[Insert Table 30: Zidio Development Mandatory Submission Deliverables Checklist] Caption: Complete submission checklist as defined in the Zidio Development Project Submission Guidelines (March 2026 Edition), with completion status for each deliverable.

#	Deliverable	Format	Evaluation Weight	Status
1	Project Documentation / Report (PDF)	1 PDF file, A4, 10–18 pages	25%	✓ Complete (this document)
2	Live Public Demo URL	HTTPS (Streamlit / Hugging Face / AWS)	30%	Week 4 (Day 25)
3	GitHub Repository	1 repo with clear folders & notebooks	20%	✓ Complete
4	README.md	Detailed, professional README in repo	15%	✓ Complete
5	Demo Video (4–8 minutes)	YouTube unlisted / Loom / Drive	10%	Day 28

Appendix D: Evaluation Criteria Mapping

[Insert Table 31: Zidio Development Evaluation Criteria Mapping] Caption: The six Zidio Development evaluation categories (100-point scale) mapped to the specific RetailPulse deliverables that address each criterion.

Category	Points	RetailPulse Evidence
Innovation & Problem Solving	15	Hybrid ensemble with optimal weight search; demand-driven inventory; SHAP-informed feature engineering
Technical Depth & Model Quality	25	Optuna 30-trial HPO; 5-fold CV; SMOTE; SHAP; $AUC \geq 0.88$; $MAPE \leq 12\%$
MLOps & Production Readiness	20	Evidently AI 0.7 drift detection; Airflow DAG with BranchPythonOperator; retraining log; model quality gate
Documentation Quality	20	14 structured notebooks; this 2-part report; inline code comments; semantic commit messages
Deployment & Reliability	10	Docker (Day 22); Kubernetes (Day 23); GitHub Actions CI/CD (Day 24); cloud deployment (Day 25)
Presentation & Polish	10	5-page Streamlit dashboard; Plotly charts; CSV/PDF export; demo video (Day 28)